

Hesaplama ve Hesaplamalı Düşünme

Bilgisayar ve Programlamaya Giriş

Hesaplama

Bilgisayar tarihi

Bilgi ve algoritma

Kontrol yapıları

Hesaplamalı düşünme

Sercan KÜLCÜ · Giresun Üniversitesi

Öğrenme Çıktıları

Hesaplama

Verinin kurallara göre işlenmesini açıklamak.

Algoritma

Bir problemi girdi, çıktı ve sonlu adımlarla tanımlamak.

Kontrol yapıları

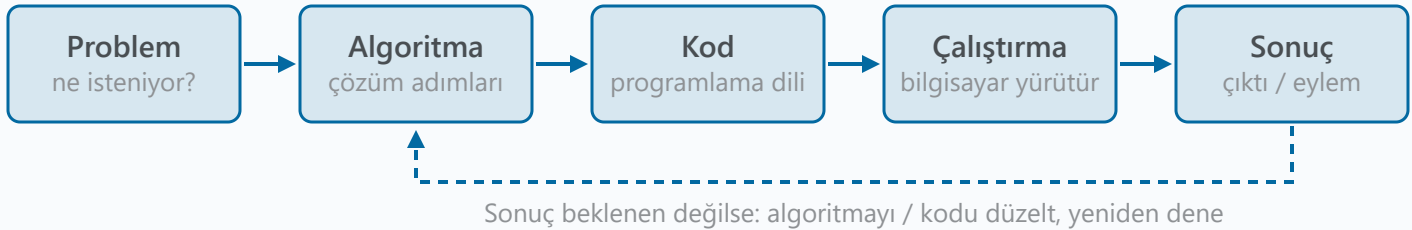
Sıralı, koşullu ve döngülü işlemleri ayırt etmek.

Hesaplamalı düşünme

Ayrıştırma, örüntü, soyutlama ve algoritma tasarımını uygulamak.

Programlama

Programlama, bir bilgisayarın gerçekleştireceği işlemleri programlama diliyle ifade etme sürecidir.



- Bilgisayar komutları eksiksiz ve kesin biçimde uygular.
- Doğru program, doğru tanımlanmış problem ve algoritmayla başlar.

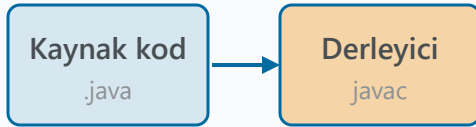
Derleyici, Yorumlayıcı ve Sanal Makine

Kaynak kod

.java

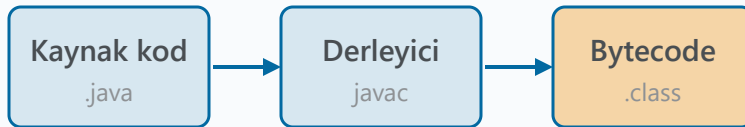
Programcı Java kaynak kodunu yazar (`Merhaba.java`). İşlemci bunu doğrudan çalıştıramaz; çevrilmesi gerekir. **Derleyici** kodu çalışmadan önce çevirir, **yorumlayıcı** çalışırken komut komut çevirip yürütür.

Derleyici, Yorumlayıcı ve Sanal Makine



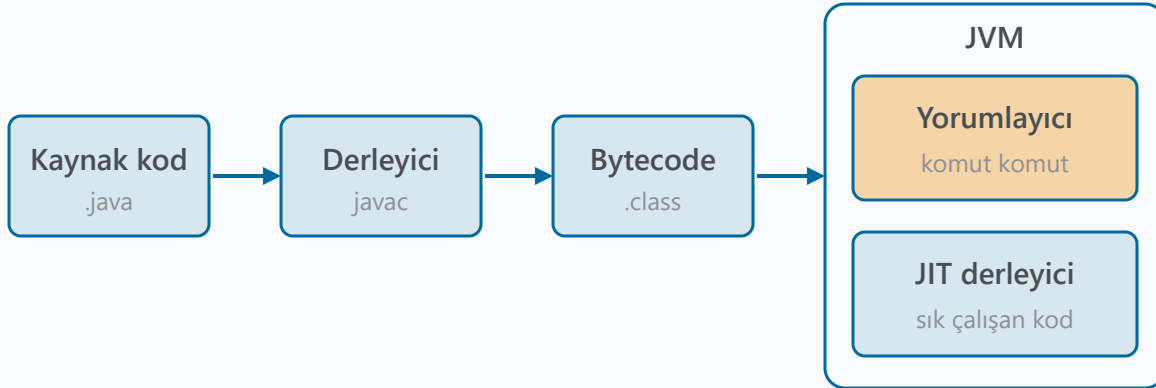
javac derleyicisi kaynak kodu denetler (söz dizimi, tip uyumu). Hata varsa süreç burada durur.

Derleyici, Yorumlayıcı ve Sanal Makine



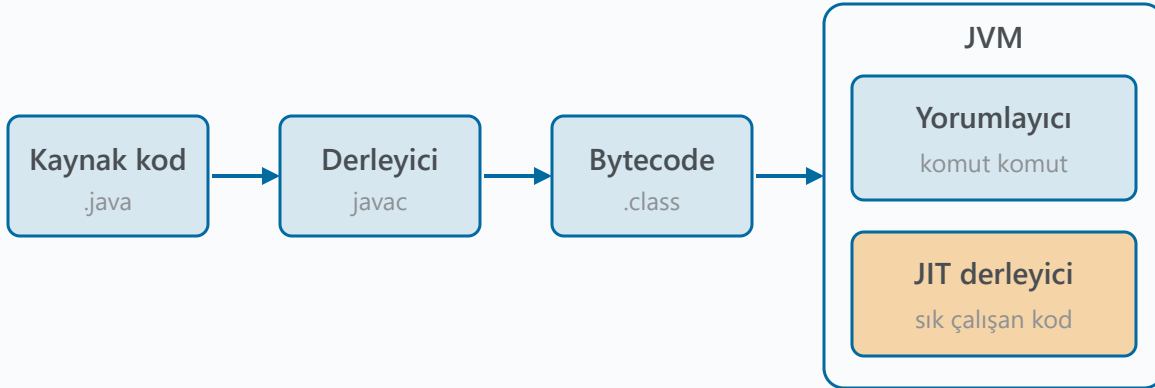
Çıktı **bytecode**'dur (`.class` dosyası): belirli bir işlemciye değil, Java Sanal Makinesi'ne (JVM) yönelik ara koddur. JVM bulunan farklı platformlarda aynı dosya çalışır.

Derleyici, Yorumlayıcı ve Sanal Makine



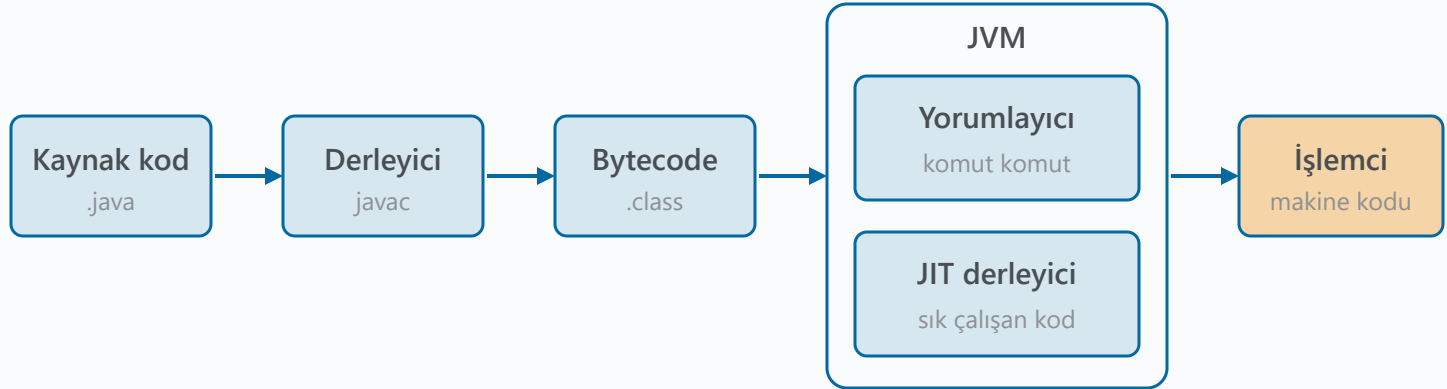
JVM bytecode'ü yürütür. Başlangıçta **yorumlayıcı** bytecode komutlarını tek tek çevirip çalıştırır.

Derleyici, Yorumlayıcı ve Sanal Makine



Tipik JVM'lerde çok sık çalışan bölümler çalışma sırasında **JIT** (tam zamanında) derleyicisiyle makine koduna çevrilir ve saklanır; sonraki çalıştırmalar hızlanır.

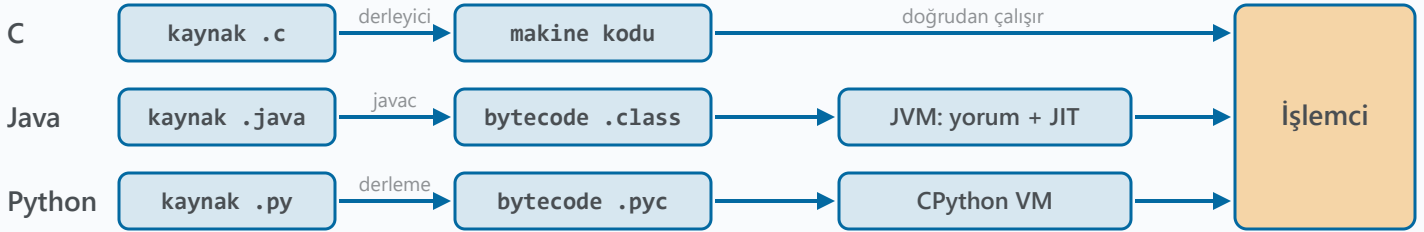
Derleyici, Yorumlayıcı ve Sanal Makine



İşlemci en sonunda yalnızca **makine komutlarını** çalıştırır. Yol: kaynak kod → bytecode → (yorumlama veya JIT) → makine kodu.

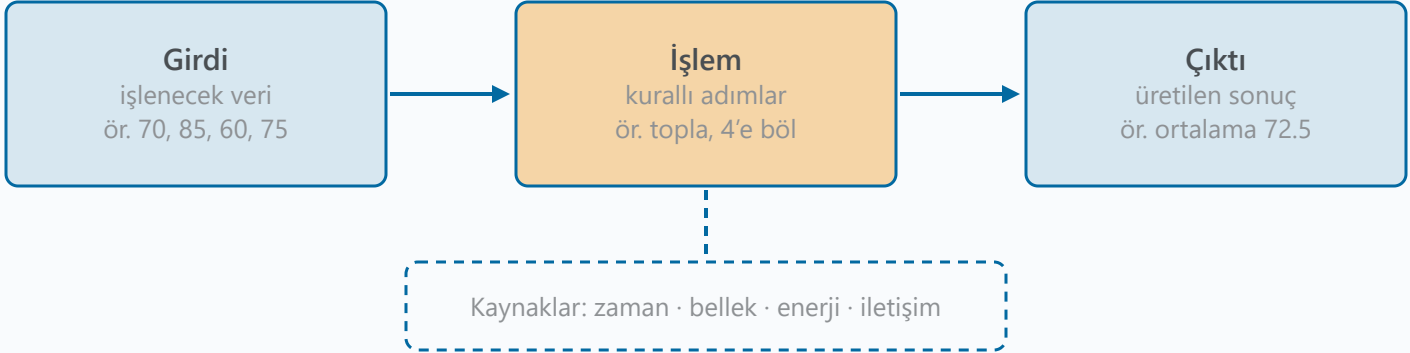
Üç Dil, Üç Çeviri Yolu

Dil	Tipik yol
C	Derleyici kaynak kodu doğrudan hedef işlemcinin makine koduna çevirir; çıkan program işlemcide çalışır.
Java	Derleyici bytecode üretir; JVM bunu yorumlar ve sık çalışan kısımları JIT ile makine koduna çevirir.
Python	Yaygın gerçekleştirmede (CPython) kaynak önce bytecode'a derlenir, sonra yorumlayıcı sanal makine yürütür.



"Derlenen" ya da "yorumlanan" ayrımı dilden çok gerçekleştirmenin özelliğidir; çoğu sistem iki yaklaşımı birlikte kullanır.

Hesaplama



Hesaplama yalnızca aritmetik değildir. Arama, sıralama, karar verme, dönüştürme ve iletişim de hesaplama.

Hesaplamanın Yapı Taşları

Matematik

Nicelikler ve ilişkiler
üzerinde işlem.

Mantık

Koşulların doğru veya yanlış
değerlendirilmesi.

Veri

Problemin temsil edilen
girdileri ve ara sonuçları.

Algoritma

İşlemlerin sonlu ve düzenli
sırası.

Donanım

Adımları fiziksel olarak
yürüten sistem.

Programlama dili

Algoritmayı bilgisayara ifade
etme aracı.

Bilgisayar Bilimi

Bilgisayar bilimi yalnızca araç kullanmayı değil, bilginin nasıl temsil edildiğini ve problemlerin nasıl çözülebileceğini inceler.

Soyutlama

Gereksiz ayrıntıları gizleyerek önemli yapıya odaklanmak.

Algoritmalar

Çözüm adımlarını tasarlamak ve analiz etmek.

Sistemler

Donanım ve yazılımın birlikte çalışmasını anlamak.

İnsan ve toplum

Teknolojinin etkilerini, güvenliğini ve etiğini değerlendirmek.

Teknoloji ve Sorumluluk

Erişilebilirlik

Farklı kullanıcıların teknolojiye eşit biçimde erişebilmesi.

Mahremiyet

Kişisel verilerin amaçla sınırlı ve güvenli kullanılması.

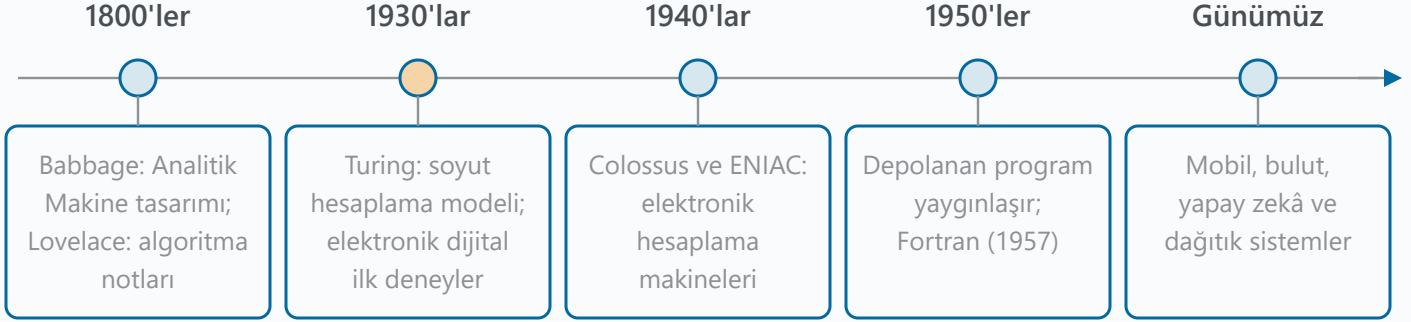
Adalet

Veri ve algoritmaların ayrımcı sonuç üretmemesi.

Sürdürülebilirlik

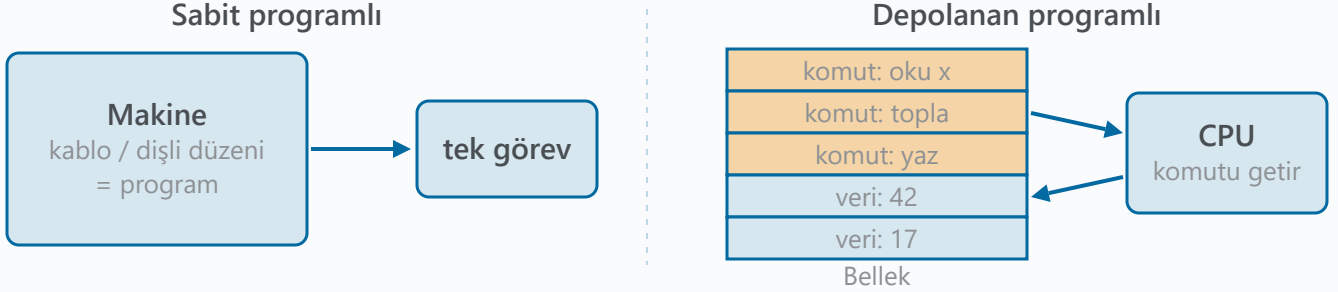
Enerji, donanım ömrü ve elektronik atığın dikkate alınması.

Hesaplama Tarihinde Dönüm Noktaları



Donanım hızlanıp küçüldükçe aynı temel fikirler (algoritma, depolanan program) daha geniş alanlarda uygulanabilir oldu.

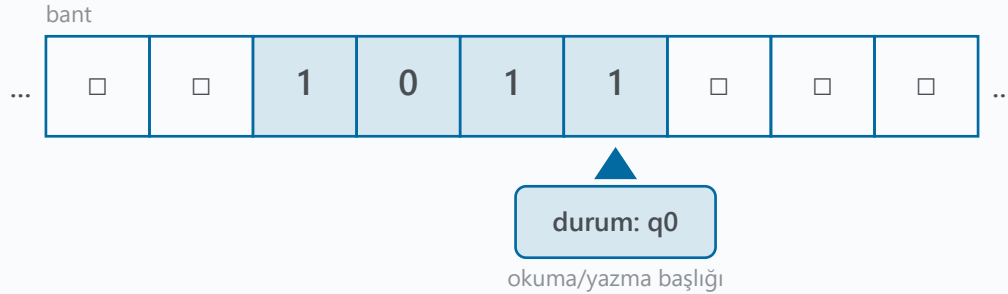
Sabit ve Depolanan Program



Yaklaşım	Program nasıl belirlenir?	Esneklik
Sabit programlı	Makinenin fiziksel düzeni veya özel mekanizması	Belirli görev
Depolanan programlı	Komutlar bellekte veri gibi saklanır	Program değiştirilerek farklı görevler
Genel amaçlı	Çok farklı programları çalıştırabilen mimari	Geniş uygulama alanı

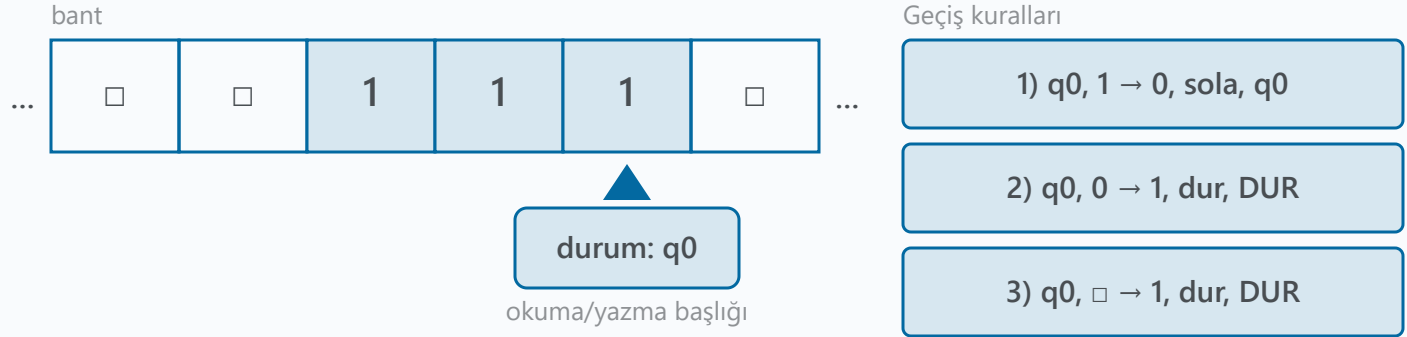
Evrensel Turing Makinesi

Turing makinesi, algoritmik hesaplamayı açıklamak için kullanılan soyut bir modeldir.



- Başlık bir simgeyi okur, yeni simge yazabilir ve sağa veya sola hareket eder.
- Geçiş kuralı mevcut durum ve simgeye göre sonraki adımı belirler.
- **Evrensel** Turing makinesi, bantta verilen başka bir makinenin kurallarını okuyup onu taklit eder: depolanan program fikrinin kuramsal temeli.

Turing Makinesi Adım Adım: 111 + 1

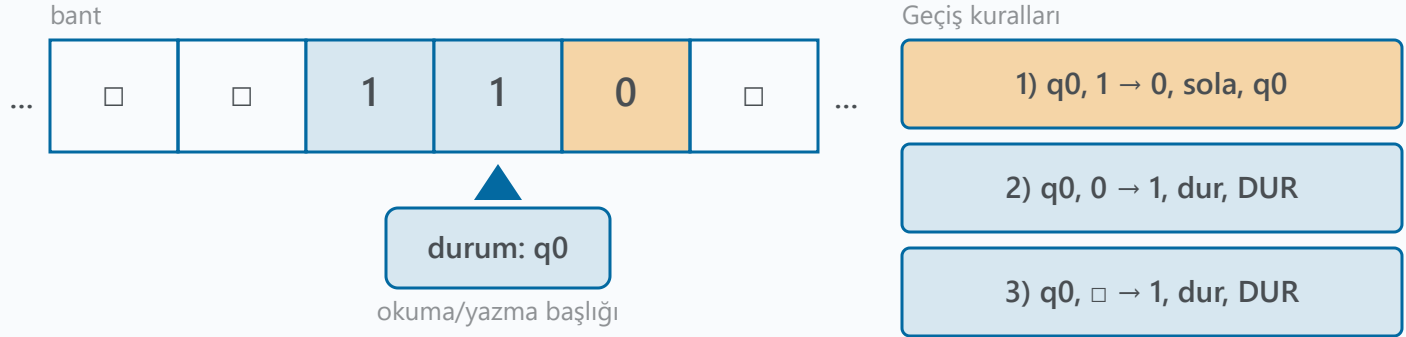


Bantta: 111 (ikili) = 7

(durum, okunan) → yaz, git, yeni durum

Başlangıç: bantta **111** (ikili 7) var, başlık en sağdaki basamakta, durum **q0**. Amaç: sayıya 1 eklemek.

Turing Makinesi Adım Adım: 111 + 1

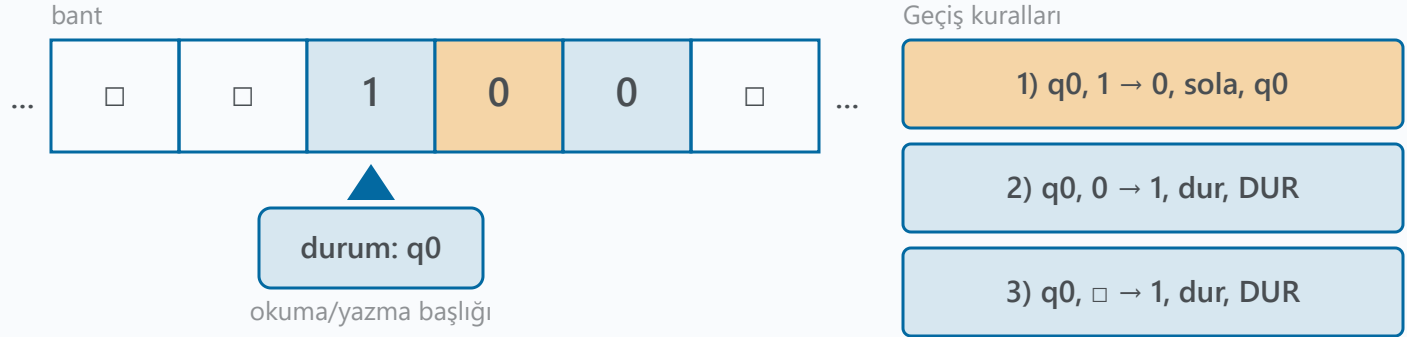


Bantta: 110 (ikili) = 6

Okunan **1**, durum q0 → kural 1: **0** yaz, başlık **sola** kayar (elde 1 devam ediyor).

(durum, okunan) → yaz, git, yeni durum

Turing Makinesi Adım Adım: 111 + 1

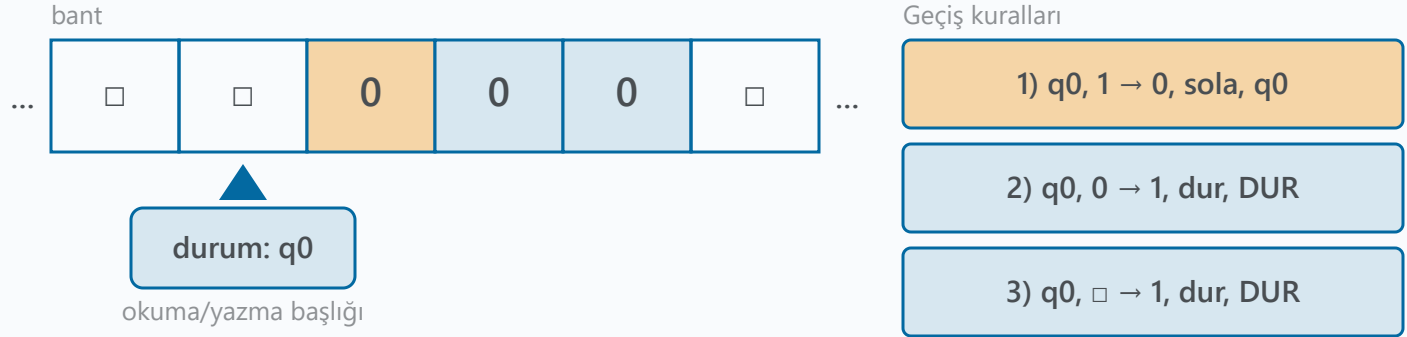


Bantta: 100 (ikili) = 4

Okunan **1**, durum q0 → kural 1: **0** yaz, başlık **sola** kayar (elde 1 devam ediyor).

(durum, okunan) → yaz, git, yeni durum

Turing Makinesi Adım Adım: 111 + 1

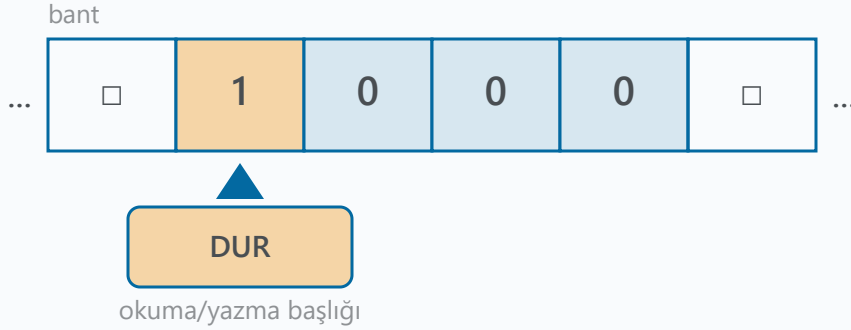


Bantta: 000 (ikili) = 0

Okunan **1**, durum q0 → kural 1: **0** yaz, başlık **sola** kayar (elde 1 devam ediyor).

(durum, okunan) → yaz, git, yeni durum

Turing Makinesi Adım Adım: 111 + 1



Geçiş kuralları

1) $q_0, 1 \rightarrow 0, \text{sola}, q_0$ 2) $q_0, 0 \rightarrow 1, \text{dur}, \text{DUR}$ 3) $q_0, \square \rightarrow 1, \text{dur}, \text{DUR}$ (durum, okunan) $\rightarrow$ yaz, git, yeni durum

Bantta: 1000 (ikili) = 8

Okunan $\square$, durum $q_0 \rightarrow$ kural 3: **1** yaz, makine **durur**. Bant: **1000** = 8. Her adımı yalnızca (durum, okunan simge) belirledi.

Adım 5 / 5

Bilgisayarların Sınırları

Zaman

Büyük girdiler bazı algoritmaları pratik olmaktan çıkarabilir.

Bellek

Veri ve ara sonuçlar sonlu depolama kullanır.

Enerji

Her işlem ve veri aktarımı enerji tüketir.

Hesaplanabilirlik

Bazı problemler için her durumda sonuç veren algoritma yoktur (ör. durma problemi).

Veri kalitesi

Eksik veya hatalı girdi doğru sonucu engeller.

Model sınırı

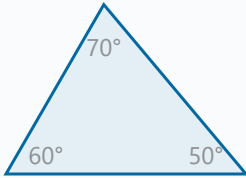
Program gerçek dünyanın yalnızca seçilmiş özelliklerini temsil eder.

Bilgi Türleri

Bildirimsel bilgi

Neyin doğru olduğunu veya bir kavramın ne olduğunu açıklar.

Bir üçgenin iç açıları toplamı 180° 'dir.



Bildirimsel: ne doğru?

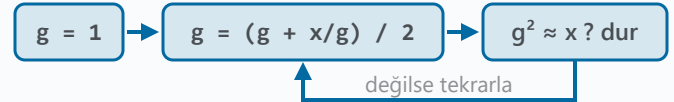
$60^\circ + 70^\circ + 50^\circ = 180^\circ$
olgu bildirir; yöntem vermez

Buyurgan (emirsel) bilgi

Bir sonuca nasıl ulaşılacağını adımlarla açıklar.

Bir sayının karekökünü yaklaşık hesapla.

Buyurgan: nasıl bulunur?



$x = 25: 1 \rightarrow 13 \rightarrow 7,46 \rightarrow 5,41 \rightarrow 5,02 \rightarrow 5,00$

Algoritmalar buyurgan bilgiyi açık, uygulanabilir ve sonlu adımlara dönüştürür.

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Karekökü aranan sayı $x = 25$.

Adım 1 / 14

► $x = 25$

```
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

$x = 25$

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Kaba bir başlangıç tahmini: **tahmin = 1**.

Adım 2 / 14

```
1 x = 25
```

```
▶ tahmin = 1
```

```
3 while |tahmin * tahmin - x| > 0.01
```

```
4     tahmin = (tahmin + x / tahmin) / 2
```

```
5 yaz(tahmin)
```

DEĞİŞKENLER

x = 25

tahmin = 1

ÇIKTI

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|1^2 - 25| = 24 \rightarrow > \mathbf{0.01}$: devam.

Adım 3 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

x = 25

tahmin = 1

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

1. tur: tahmin = $(1 + 25 / 1) / 2 = 13$. Tahmin ile x/tahmin 'in ortalaması alınır.

Adım 4 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

x = 25

tahmin = 13

ÇIKTI

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|13^2 - 25| = 144 \rightarrow > 0.01$: devam.

Adım 5 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

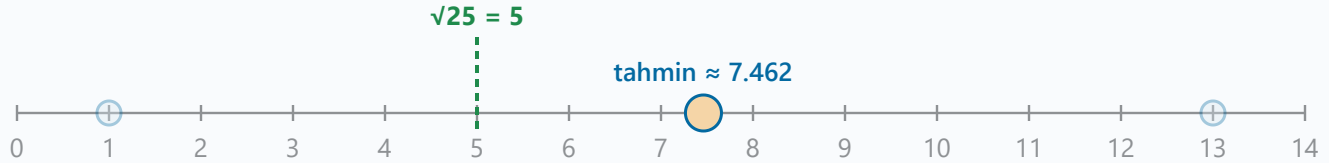
x = 25

tahmin = 13

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

2. tur: tahmin = $(13 + 25 / 13) / 2 = 7.462$. Tahmin ile x/tahmin 'in ortalaması alınır.

Adım 6 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

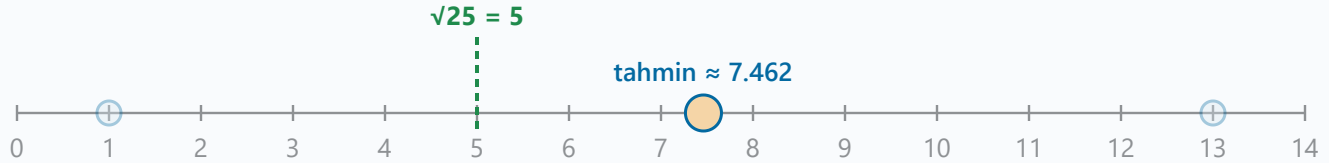
x = 25

tahmin = 7.4615

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|7.462^2 - 25| = 30.67 \rightarrow > \mathbf{0.01}$: devam.

Adım 7 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

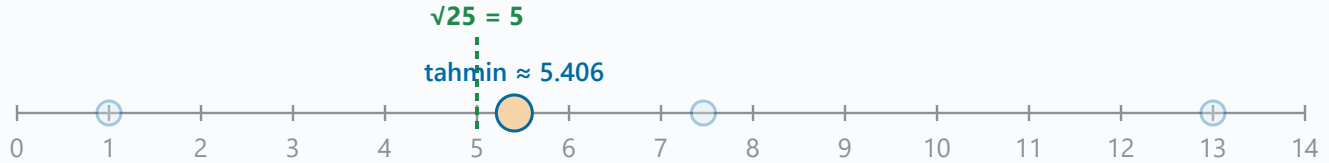
x = 25

tahmin = 7.4615

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

3. tur: $\text{tahmin} = (7.462 + 25 / 7.462) / 2 = \mathbf{5.406}$. Tahmin ile x/tahmin 'in ortalaması alınır.

Adım 8 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

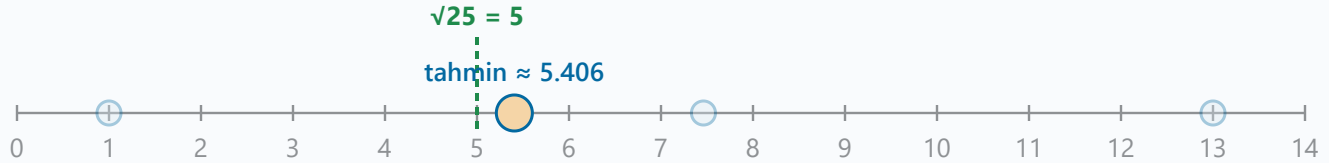
$x = 25$

$\text{tahmin} = 5.406$

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Soluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|5.406^2 - 25| = 4.225 \rightarrow > \mathbf{0.01}$: devam.

Adım 9 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

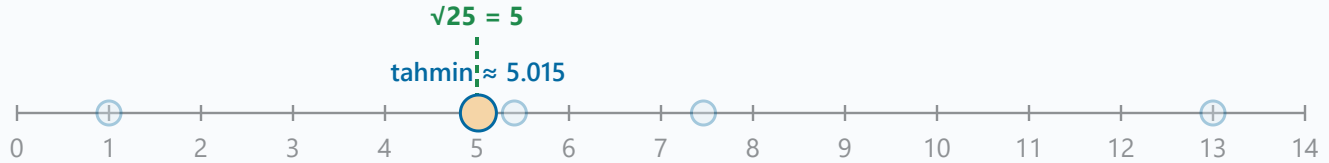
x = 25

tahmin = 5.406

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

4. tur: $\text{tahmin} = (5.406 + 25 / 5.406) / 2 = \mathbf{5.015}$. Tahmin ile x/tahmin 'in ortalaması alınır.

Adım 10 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

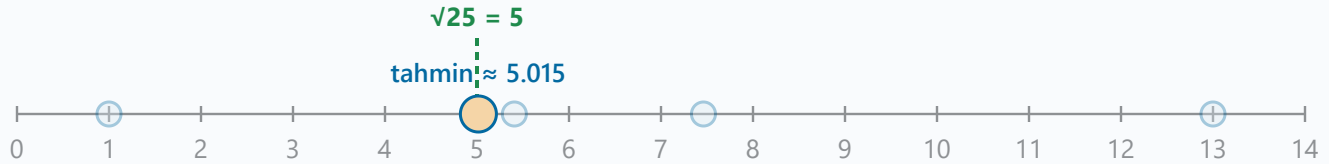
DEĞİŞKENLER

$x = 25$

$\text{tahmin} = 5.0152$

ÇIKTI

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|5.015^2 - 25| = 0.1527 \rightarrow > \mathbf{0.01}$: devam.

Adım 11 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

DEĞİŞKENLER

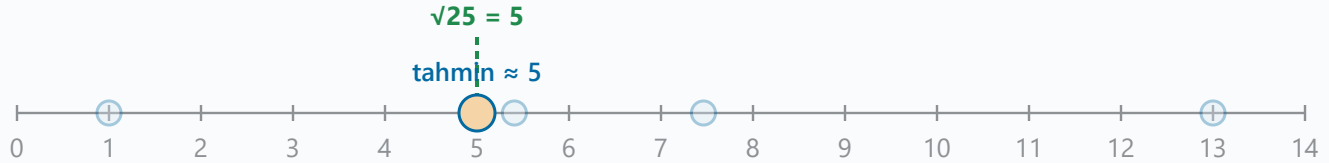
x = 25

tahmin = 5.0152

ÇIKTI



Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

5. tur: $\text{tahmin} = (5.015 + 25 / 5.015) / 2 = 5$. Tahmin ile x/tahmin 'in ortalaması alınır.

Adım 12 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

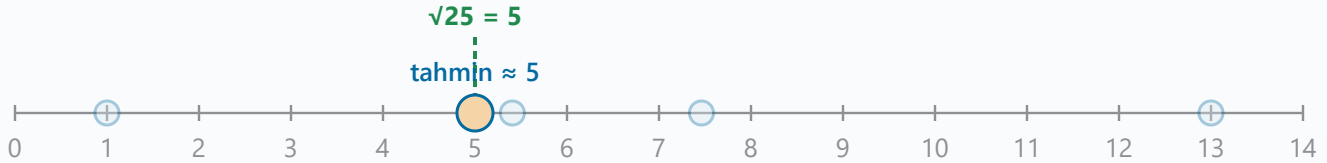
DEĞİŞKENLER

x = 25

tahmin = 5

ÇIKTI

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Hata $|5^2 - 25| = 0.0002318 \rightarrow \leq \mathbf{0.01}$: **yeterince yakın, dur.**

Adım 13 / 14

```
1 x = 25
2 tahmin = 1
▶ while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
5 yaz(tahmin)
```

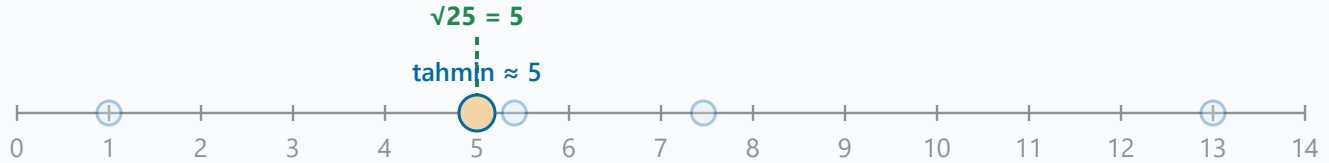
DEĞİŞKENLER

x = 25

tahmin = 5

ÇIKTI

Buyurgan Bilgi: Karekökü Yaklaşık Bulma ($x = 25$)



Suluk noktalar: önceki tahminler. Her tur tahmini $\sqrt{x}$ 'e yaklaştırır.

Sonuç yazılır: $\approx$ **5.0000**. 5 turda gerçek değere (5) çok yaklaştı: "nasıl bulunur" bilgisi = **buyurgan bilgi**.

Adım 14 / 14

```
1 x = 25
2 tahmin = 1
3 while |tahmin * tahmin - x| > 0.01
4     tahmin = (tahmin + x / tahmin) / 2
▶ yaz(tahmin)
```

DEĞİŞKENLER

x = 25

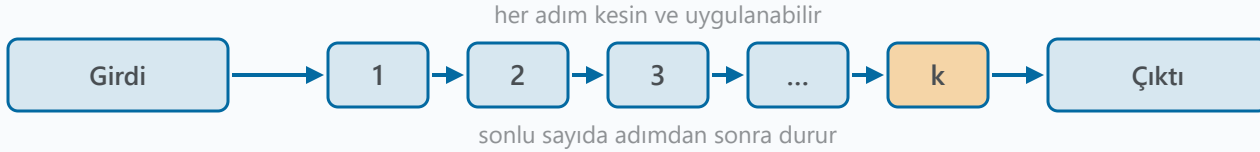
tahmin = 5

ÇIKTI

5.0000

Algoritma

Algoritma, belirli bir problemi çözmek veya görevi tamamlamak için tanımlanmış sonlu adımlar dizisidir.



Girdi

Sıfır veya daha fazla veri alabilir.

Çıktı

En az bir sonuç üretir.

Kesinlik

Her adım açık ve uygulanabilir olmalıdır.

Sonluluk

Belirli sayıda adımdan sonra durmalıdır.

Doğruluk

Geçerli girdiler için beklenen sonucu vermelidir.

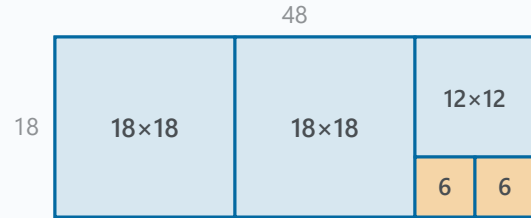
Verimlilik

Zaman ve belleği makul kullanılmalıdır.

Öklid Algoritması

İki pozitif tam sayının en büyük ortak bölenini tekrar eden kalan işlemleriyle bulur.

```
GCD(a, b)
while b ≠ 0
    r = a mod b
    a = b
    b = r
return a
```



$$48 = 2 \cdot 18 + 12 \cdot 18 = 1 \cdot 12 + 6 \cdot 12 = 2 \cdot 6$$

Her şeyi kalansız döşeyen kare: 6×6

$$\text{GCD}(48, 18): 48 \bmod 18 = 12 \cdot 18 \bmod 12 = 6 \cdot 12 \bmod 6 = 0 \cdot \text{sonuç } 6$$

Öklid Adım Adım: GCD(48, 18)

a = 48

48

b = 18

18

GCD(48, 18) çağrılır: a = 48, b = 18. Çubuklar sayıların uzunluğudur.

Adım 1 / 15

► GCD(a, b)

```
2 while b ≠ 0
3   r = a mod b
4   a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 48

b = 18

Öklid Adım Adım: GCD(48, 18)

a = 48

48

b = 18

18

b ≠ 0 → 18 ≠ 0 → **doğru**: 1. tur başlar.

Adım 2 / 15

```
1 GCD(a, b)
```

```
▶ while b ≠ 0
```

```
3   r = a mod b
```

```
4   a = b
```

```
5   b = r
```

```
6 return a
```

DEĞİŞKENLER

a = 48

b = 18

Öklid Adım Adım: GCD(48, 18)

a = 48

18

18

r=12

b = 18

18

$$48 = 2 \times 18 + 12 \rightarrow \text{kalan } r = 12$$

r = a mod b = 48 mod 18 = **12**: a'dan b uzunluğunda 2 parça çıkar, 12 kalır.

Adım 3 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   ► r = a mod b
4   a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 48

b = 18

r = 12

Öklid Adım Adım: GCD(48, 18)

a = 18

18

b = 18

18

a = b → a = 18. Artık küçük sayı bölünen olur.

Adım 4 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   r = a mod b
4   ▶ a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 18

b = 18

r = 12

Öklid Adım Adım: GCD(48, 18)

a = 18

18

b = 12

12

b = r → b = 12. Ortak bölen, kalanı da böler; problem küçüldü.

Adım 5 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   r = a mod b
4   a = b
5   ▶ b = r
6 return a
```

DEĞİŞKENLER

a = 18

b = 12

r = 12

Öklid Adım Adım: GCD(48, 18)

a = 18

18

b = 12

12

b ≠ 0 → 12 ≠ 0 → **doğru**: 2. tur başlar.

Adım 6 / 15

```
1 GCD(a, b)
```

```
▶ while b ≠ 0
```

```
3   r = a mod b
```

```
4   a = b
```

```
5   b = r
```

```
6 return a
```

DEĞİŞKENLER

a = 18

b = 12

r = 12

Öklid Adım Adım: GCD(48, 18)

a = 18 12 r=6

b = 12 12

$$18 = 1 \times 12 + 6 \rightarrow \text{kalan } r = 6$$

r = a mod b = 18 mod 12 = **6**: a'dan b uzunluğunda 1 parça çıkar, 6 kalır.

Adım 7 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   ► r = a mod b
4   a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 18

b = 12

r = 6

Öklid Adım Adım: GCD(48, 18)

a = 12

12

b = 12

12

a = b → a = 12. Artık küçük sayı bölünen olur.

Adım 8 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   r = a mod b
4   ▶ a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 12

b = 12

r = 6

Öklid Adım Adım: GCD(48, 18)

a = 12

12

b = 6

6

b = r → b = 6. Ortak bölen, kalanı da böler; problem küçüldü.

Adım 9 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3     r = a mod b
4     a = b
5     ► b = r
6 return a
```

DEĞİŞKENLER

a = 12

b = 6

r = 6

Öklid Adım Adım: GCD(48, 18)

a = 12

12

b = 6

6

b ≠ 0 → $6 \neq 0$ → **doğru**: 3. tur başlar.

Adım 10 / 15

```
1 GCD(a, b)
```

```
▶ while b ≠ 0
```

```
3   r = a mod b
```

```
4   a = b
```

```
5   b = r
```

```
6 return a
```

DEĞİŞKENLER

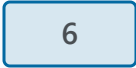
a = 12

b = 6

r = 6

Öklid Adım Adım: GCD(48, 18)

a = 12 

b = 6 

$$12 = 2 \times 6 + 0 \rightarrow \text{kalan } r = 0$$

r = a mod b = 12 mod 6 = **0**: a'dan b uzunluğunda 2 parça çıkar, 0 kalır.

Adım 11 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   ► r = a mod b
4   a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 12

b = 6

r = 0

Öklid Adım Adım: GCD(48, 18)

a = 6

6

b = 6

6

a = b → a = 6. Artık küçük sayı bölünen olur.

Adım 12 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3   r = a mod b
4   ▶ a = b
5   b = r
6 return a
```

DEĞİŞKENLER

a = 6

b = 6

r = 0

Öklid Adım Adım: GCD(48, 18)

$a = 6$

6

$b = 0$ (0: bölünecek bir şey kalmadı)

$b = r \rightarrow b = 0$. Ortak bölen, kalanı da böler; problem küçüldü.

Adım 13 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3     r = a mod b
4     a = b
5     ▶ b = r
6 return a
```

DEĞİŞKENLER

$a = 6$

$b = 0$

$r = 0$

Öklid Adım Adım: GCD(48, 18)

a = 6

6

b = 0 (0: bölünecek bir şey kalmadı)

b ≠ 0 → $0 \neq 0$ → **yanlış**: döngü biter.

Adım 14 / 15

```
1 GCD(a, b)
```

```
▶ while b ≠ 0
```

```
3   r = a mod b
```

```
4   a = b
```

```
5   b = r
```

```
6 return a
```

DEĞİŞKENLER

a = 6

b = 0

r = 0

Öklid Adım Adım: GCD(48, 18)

a = 6

6

b = 0 (0: bölünecek bir şey kalmadı)

return a → 6. GCD(48, 18) = 6: 48 ve 18, 6 uzunluğundaki parçalarla tam ölçülür. 3 turda bitti.

Adım 15 / 15

```
1 GCD(a, b)
2 while b ≠ 0
3     r = a mod b
4     a = b
5     b = r
▶ return a
```

DEĞİŞKENLER

a = 6

b = 0

r = 0

Problem Tanımı

Program yazmadan önce problemin sınırları ve beklenen davranışı açıkça belirlenir.

Amaç

Hangi sorunun çözüleceği.

Girdi

Hangi veri ve koşulların verildiği.

Çıktı

Hangi sonucun ve biçimin beklendiği.

Kısıtlar

Boyut, süre, doğruluk ve kaynak sınırları.

Örnekler

Beklenen davranışı somutlaştıran durumlar.

Hata durumları

Geçersiz veya eksik girdide yapılacak işlem.

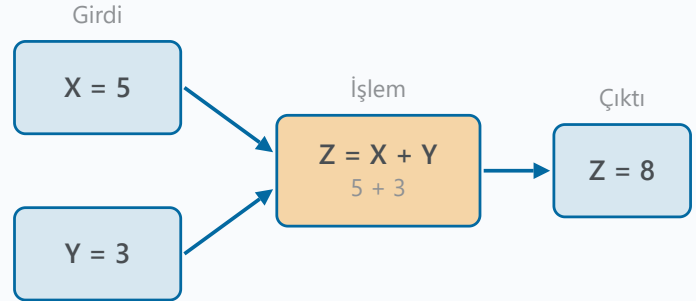
Girdi ve Çıktı Örneği

Problem

Verilen iki sayının toplamını bul.

Girdi: X ve Y sayıları

Çıktı: $Z = X + Y$

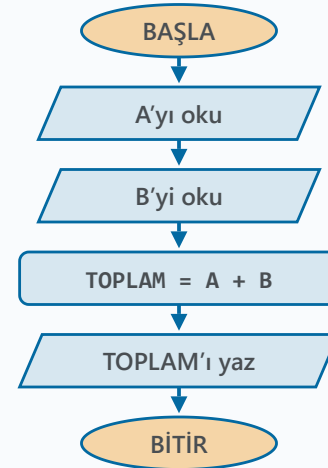


Örnek girdi-çıktı çiftleri, algoritmayı yazmadan önce beklenen davranışı netleştirir ve daha sonra test için kullanılır.

Sözde Kod

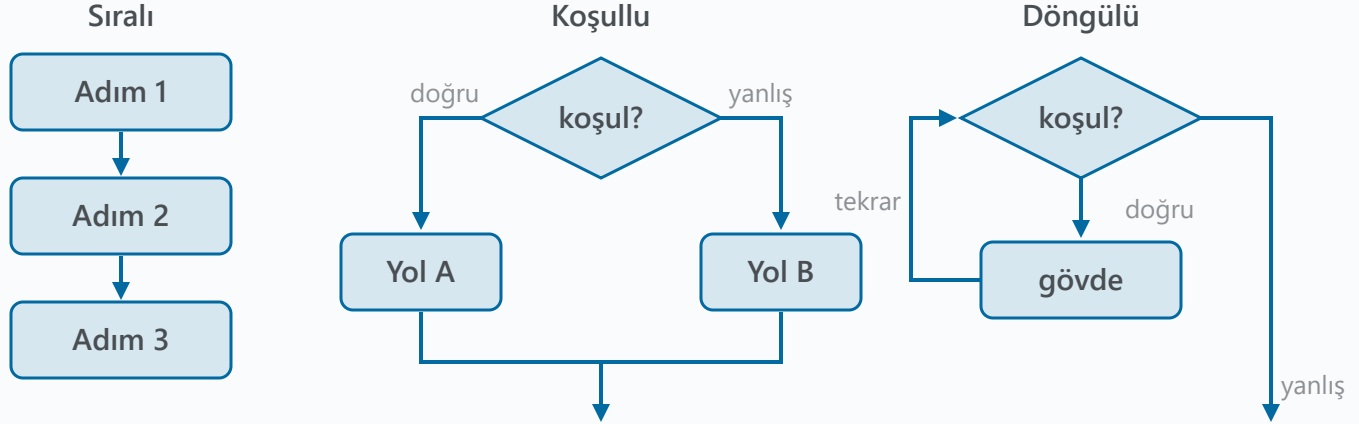
Sözde kod, algoritmanın mantığını belirli bir programlama diline bağlı kalmadan açıklar.

```
BAŞLA  
  A değerini oku  
  B değerini oku  
  TOPLAM = A + B  
  TOPLAM değerini yaz  
BİTİR
```



- Doğal dile yakın, kısa ve tutarlı ifadeler kullanılır.
- Girinti, koşul ve döngülerin kapsamını gösterir.
- Amaç söz dizimi değil, algoritmayı anlaşılır hale getirmektir.

Temel Kontrol Yapıları



Sıralı

Adımlar yazıldığı sırada bir kez.

Koşullu

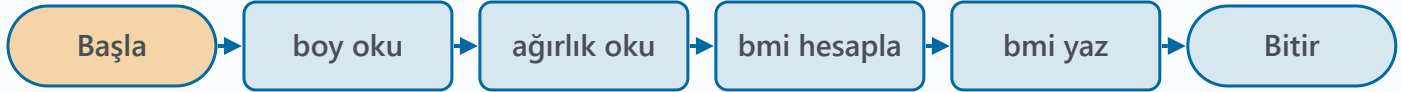
Koşula göre tek bir yol seçilir.

Döngülü

Koşul doğru oldukça tekrar.

Bu üç yapı birlikte her algoritmanın akışını ifade etmek için temel bir araç takımı oluşturur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

Algoritma **Başla**'dan girer. Örnek girdi: boy = 1.75 m, ağırlık = 70 kg.

Adım 1 / 6

```
1 boy = oku()
2 ağırlık = oku()
3 bmi = ağırlık / (boy * boy)
4 yaz(bmi)
```

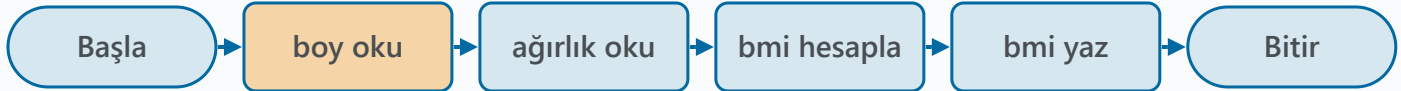
DEĞİŞKENLER

—

ÇIKTI

Sıra önemlidir: `bmi`, `boy` ve `ağırlık` okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

Kullanıcıdan boy okunur: **boy = 1.75**.

Adım 2 / 6

```
► boy = oku()  
2 ağırlık = oku()  
3 bmi = ağırlık / (boy * boy)  
4 yaz(bmi)
```

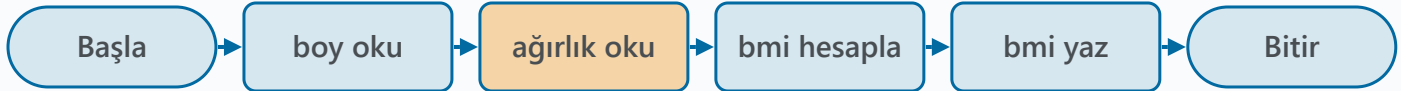
DEĞİŞKENLER

boy = 1.75

ÇIKTI

Sıra önemlidir: `bmi`, `boy` ve `ağırlık` okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

Ağırlık okunur: **agirlik = 70**.

Adım 3 / 6

```
1 boy = oku()  
▶ agirlik = oku()  
3 bmi = agirlik / (boy * boy)  
4 yaz(bmi)
```

DEĞİŞKENLER

boy = 1.75

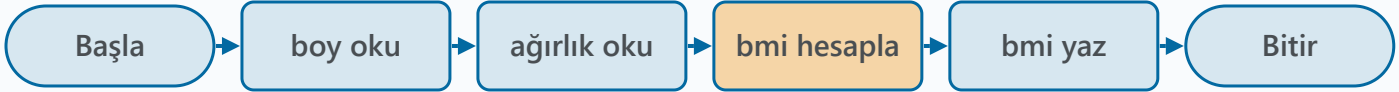
agirlik = 70

ÇIKTI



Sıra önemlidir: `bmi`, `boy` ve `ağırlık` okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

bmi = 70 / (1.75 × 1.75) = 70 / 3.0625 ≈ 22.86. Önce parantez içi hesaplanır.

Adım 4 / 6

```
1 boy = oku()
2 ağırlık = oku()
▶ bmi = ağırlık / (boy * boy)
4 yaz(bmi)
```

DEĞİŞKENLER

boy = 1.75

ağırlık = 70

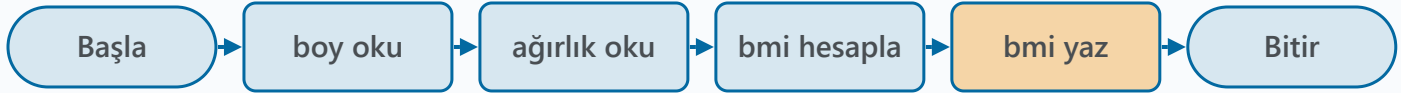
bmi = 22.86

ÇIKTI



Sıra önemlidir: **bmi** , boy ve ağırlık okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

Sonuç ekrana yazılır.

Adım 5 / 6

```
1 boy = oku()
2 ağırlık = oku()
3 bmi = ağırlık / (boy * boy)
▶ yaz(bmi)
```

DEĞİŞKENLER

boy = 1.75

ağırlık = 70

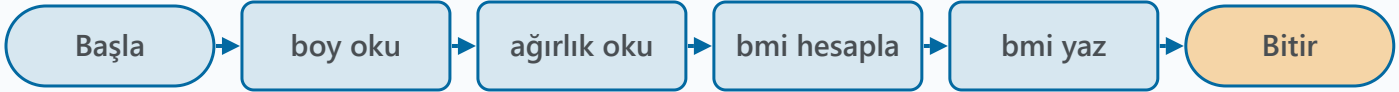
bmi = 22.86

ÇIKTI

22.86

Sıra önemlidir: `bmi`, `boy` ve `ağırlık` okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Sıralı İşlemler: BMI Hesabı



Sıralı akış: her kutu bir kez, soldan sağa

Bitir: dört adımın her biri **tam bir kez** ve yazıldığı sırayla çalıştı. Sıra değişseydi (ör. önce hesap) sonuç yanlış olurdu.

Adım 6 / 6

```
1 boy = oku()
2 agirlik = oku()
3 bmi = agirlik / (boy * boy)
4 yaz(bmi)
```

DEĞİŞKENLER

boy = 1.75

agirlik = 70

bmi = 22.86

ÇIKTI

22.86

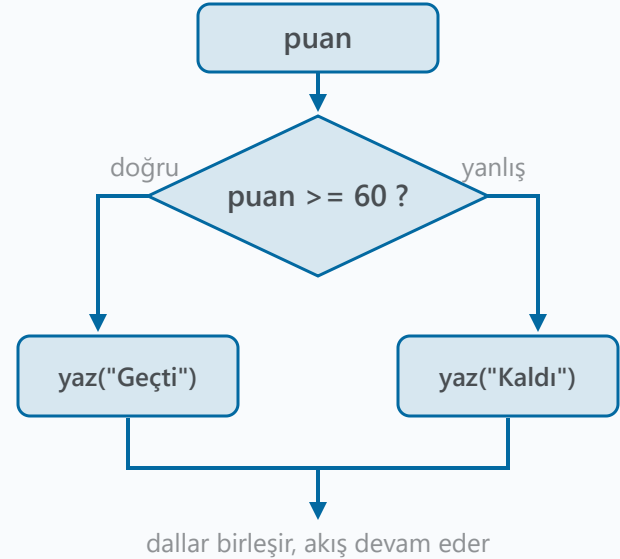
Sıra önemlidir: `bmi`, `boy` ve `ağırlık` okunmadan hesaplanamaz. Sıralı yapıda atlama ya da tekrar yoktur.

Koşullu İşlemler

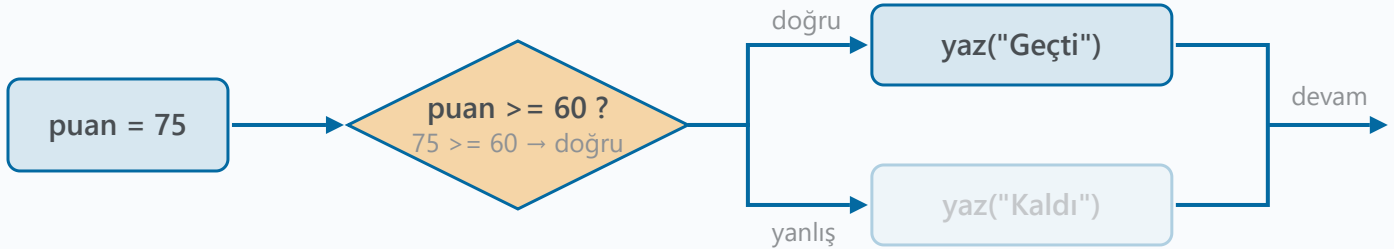
Koşullu yapıda koşul doğruysa bir yol, yanlışsa başka bir yol yürütülür.

```
eğer puan >= 60 ise  
    yaz("Geçti")  
aksi halde  
    yaz("Kaldı")
```

- **Koşul:** doğru ya da yanlış sonuç veren ifade.
- **Dallar:** her sonuç için bir işlem yolu; yalnızca biri çalışır.
- **Kapsam:** girinti, hangi adımın hangi dalda olduğunu gösterir.



Koşullu İşlem Adım Adım: 75, 40, 60



Girdi: **puan = 75**. Koşul değerlendirilir: **75 >= 60** → **doğru**.

Adım 1 / 7

► **eğer** puan >= 60 **ise**

2 yaz("Geçti")

3 **aksi halde**

4 yaz("Kaldı")

DEĞİŞKENLER

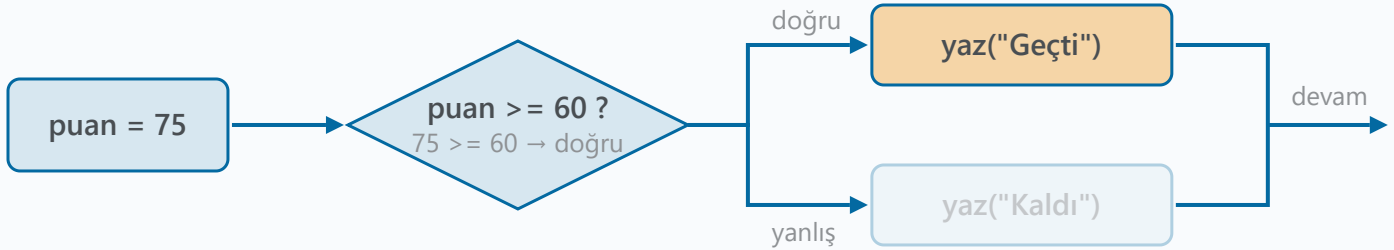
puan = **75**

koşul = **doğru**

ÇIKTI



Koşullu İşlem Adım Adım: 75, 40, 60



"Doğru" dalı çalışır; **aksi halde** kısmı atlanır.

Adım 2 / 7

```
1 eğer puan >= 60 ise
▶ yaz("Geçti")
3 aksi halde
4 yaz("Kaldı")
```

DEĞİŞKENLER

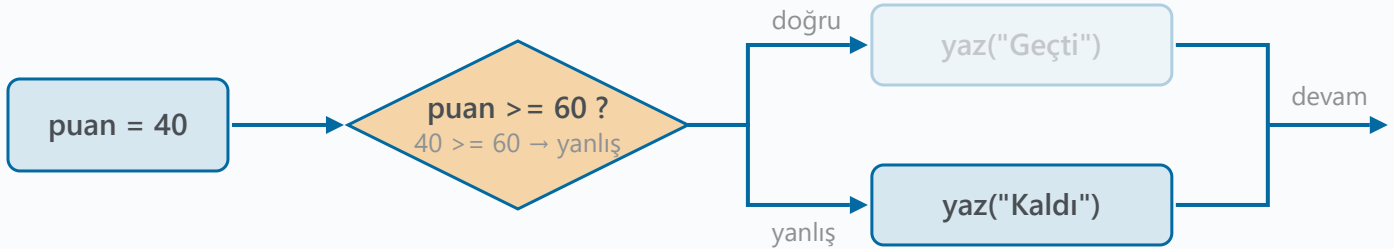
puan = 75

koşul = doğru

ÇIKTI

Geçti

Koşullu İşlem Adım Adım: 75, 40, 60



Yeni girdi: **puan = 40**. Koşul değerlendirilir: **40 >= 60** → **yanlış**.

Adım 3 / 7

► **eğer** puan >= 60 **ise**

2 yaz("Geçti")

3 **aksi halde**

4 yaz("Kaldı")

DEĞİŞKENLER

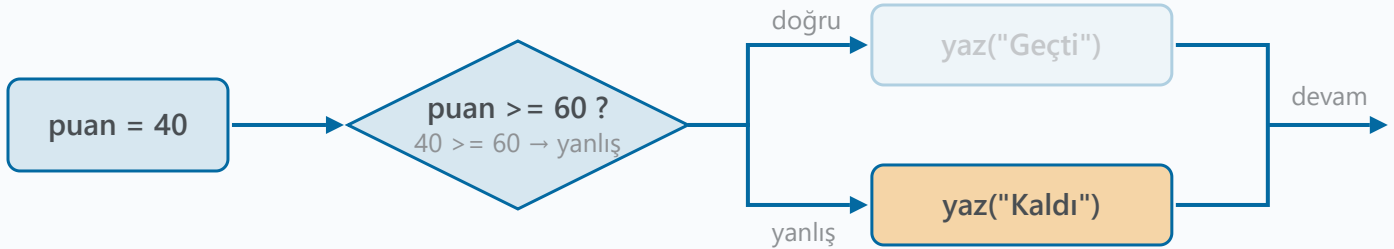
puan = 40

koşul = yanlış

ÇIKTI



Koşullu İşlem Adım Adım: 75, 40, 60



Koşul yanlış olduğu için ilk dal atlanır, **aksi halde** dalına geçilir.

Adım 4 / 7

```
1 eğer puan >= 60 ise
2   yaz("Geçti")
▶ aksi halde
4   yaz("Kaldı")
```

DEĞİŞKENLER

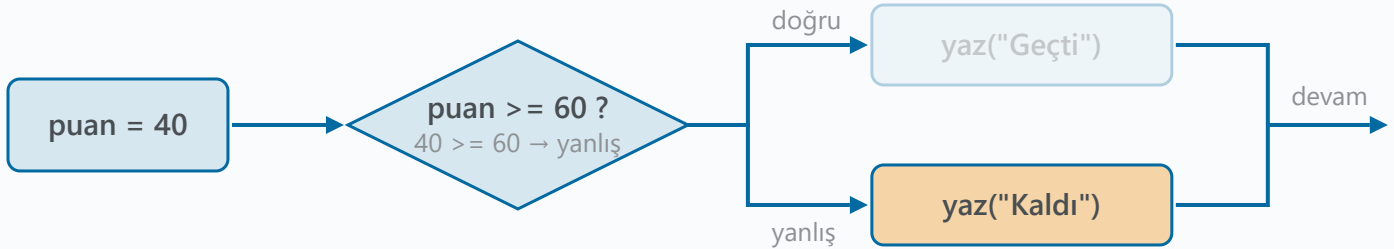
puan = 40

koşul = yanlış

ÇIKTI



Koşullu İşlem Adım Adım: 75, 40, 60



"Yanlış" dalı çalışır. Her girdide iki daldan **yalnızca biri** yürütülür.

Adım 5 / 7

```
1 eğer puan >= 60 ise
2   yaz("Geçti")
3 aksi halde
  ► yaz("Kaldı")
```

DEĞİŞKENLER

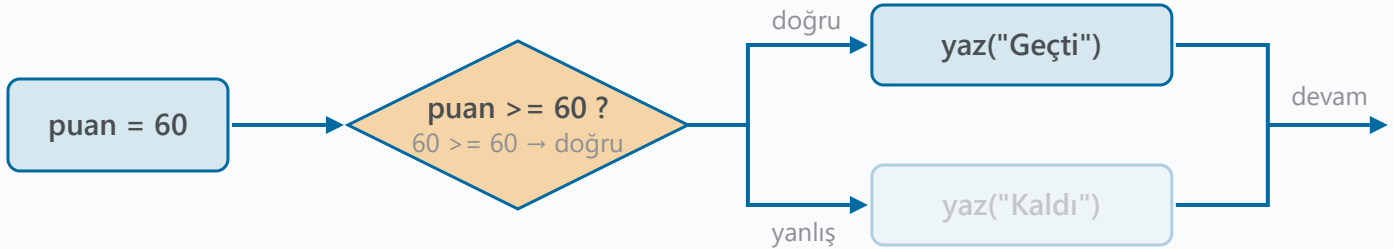
puan = 40

koşul = yanlış

ÇIKTI

Kaldı

Koşullu İşlem Adım Adım: 75, 40, 60



Yeni girdi: **puan = 60**. Koşul değerlendirilir: **60 >= 60** → **doğru** (eşitlik de "büyük eşit" kapsamında).

Adım 6 / 7

► **eğer** puan >= 60 **ise**

2 yaz("Geçti")

3 **aksi halde**

4 yaz("Kaldı")

DEĞİŞKENLER

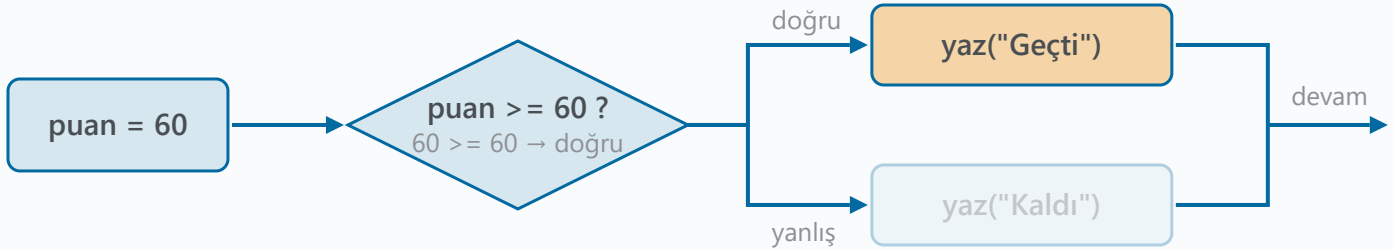
puan = 60

koşul = doğru

ÇIKTI



Koşullu İşlem Adım Adım: 75, 40, 60



"Doğru" dalı çalışır; **aksi halde** kısmı atlanır.

Adım 7 / 7

```
1 eğer puan >= 60 ise
▶ yaz("Geçti")
3 aksi halde
4 yaz("Kaldı")
```

DEĞİŞKENLER

puan = 60

koşul = doğru

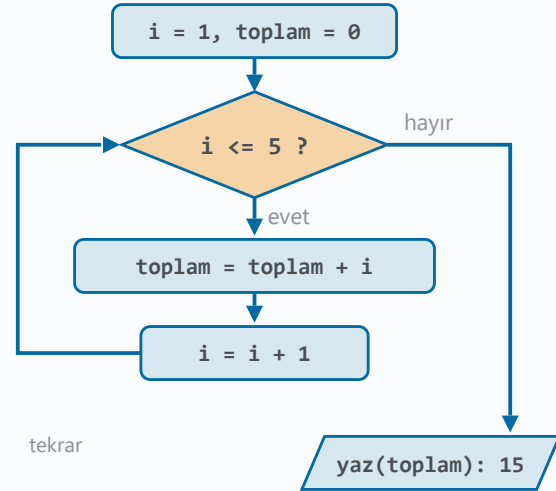
ÇIKTI

Geçti

Döngülü İşlemler

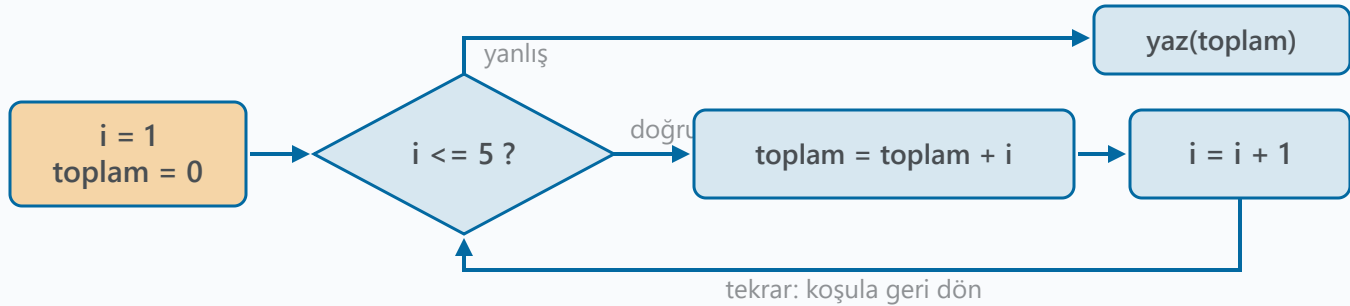
Döngü, devam koşulu doğru olduğu sürece gövdesindeki adımları tekrarlar.

```
i = 1
toplam = 0
while i <= 5
    toplam = toplam + i
    i = i + 1
yaz(toplam)
```



Döngü değişkeni ilerlemiyorsa devam koşulu hiç yanlış olmayabilir ve sonsuz döngü oluşabilir.

Döngü Adım Adım: 1'den 5'e Toplam



Sayaç $i = 1$ ile başlar.

Adım 1 / 19

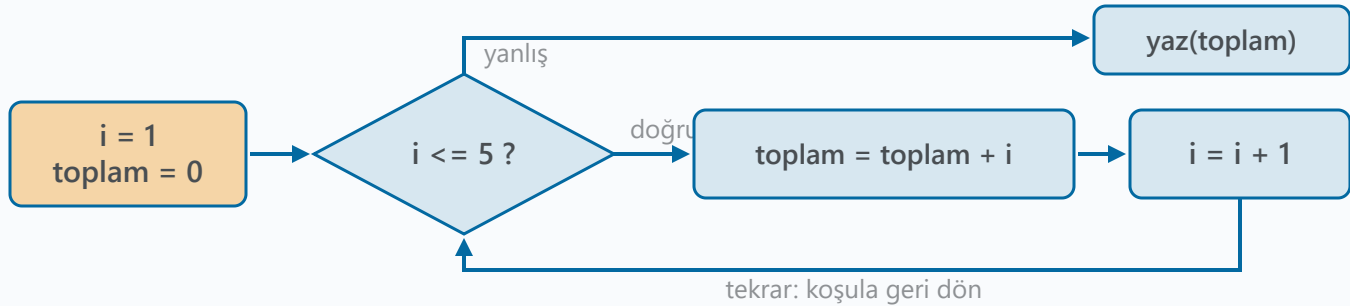
```
1 ► i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

$i = 1$

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Biriktirici değişken **toplam** = 0.

Adım 2 / 19

```
1 i = 1
2 ► toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

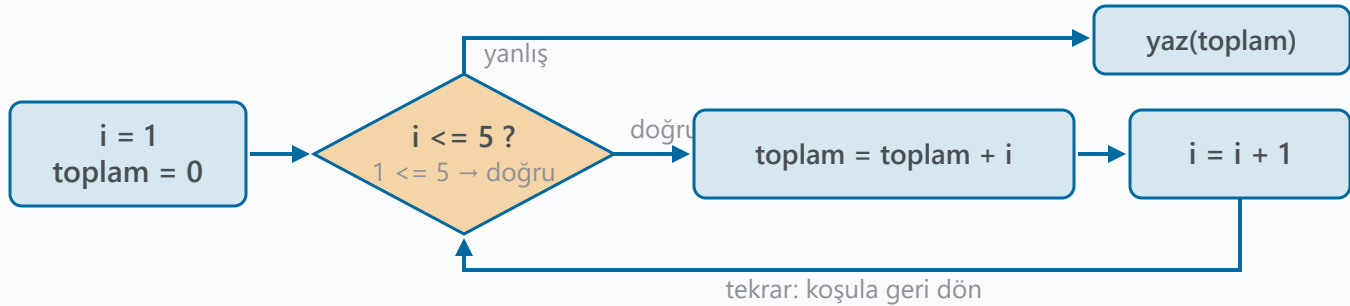
DEĞİŞKENLER

i = 1

toplam = 0

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: **1 <= 5** → **doğru**, 1. tur.

Adım 3 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

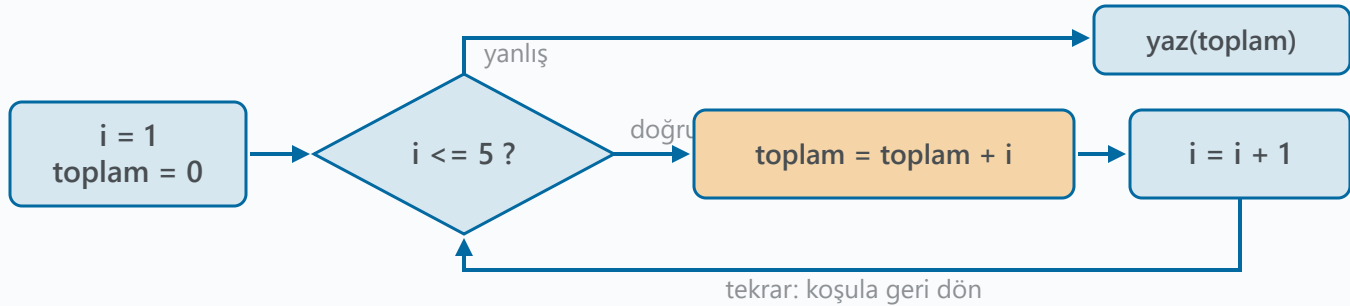
DEĞİŞKENLER

i = 1

toplam = 0

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



toplam = 0 + 1 = 1

Adım 4 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

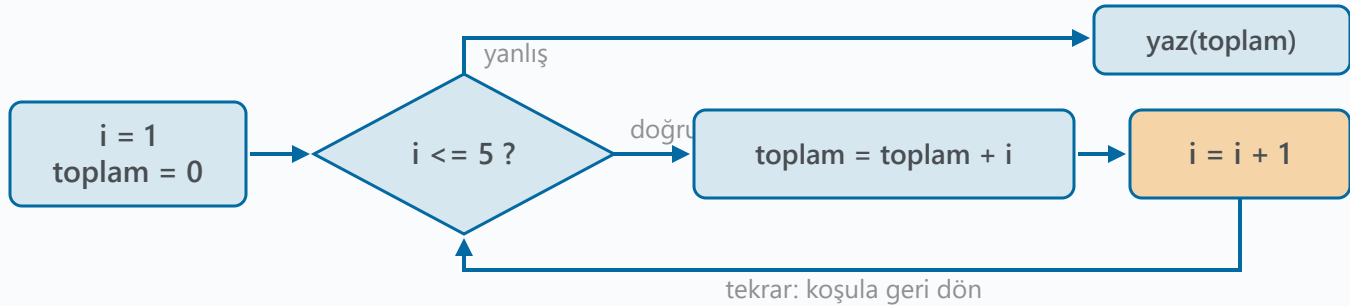
DEĞİŞKENLER

i = 1

toplam = 1

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



`i = 1 + 1 = 2`: sayaç ilerler; bu adım olmasa döngü hiç bitmezdi.

Adım 5 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

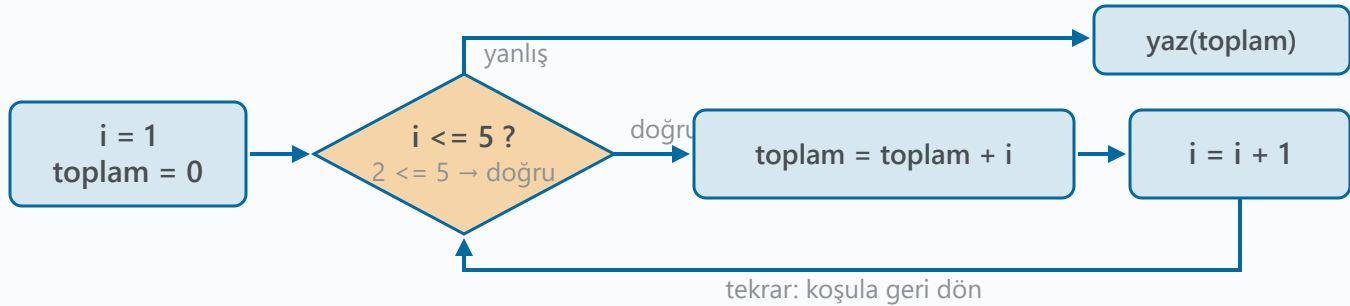
DEĞİŞKENLER

`i = 2`

`toplam = 1`

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: `2 <= 5` → **doğru**, 2. tur.

Adım 6 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

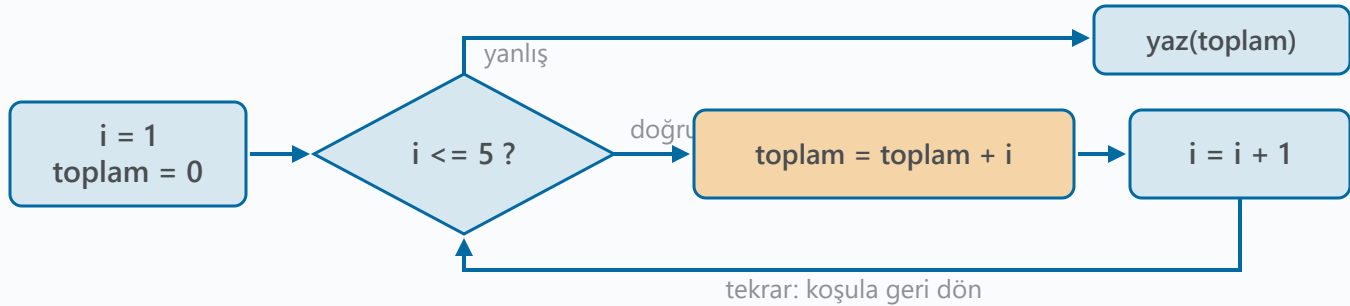
i = 2

toplam = 1

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



toplam = 1 + 2 = 3

Adım 7 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

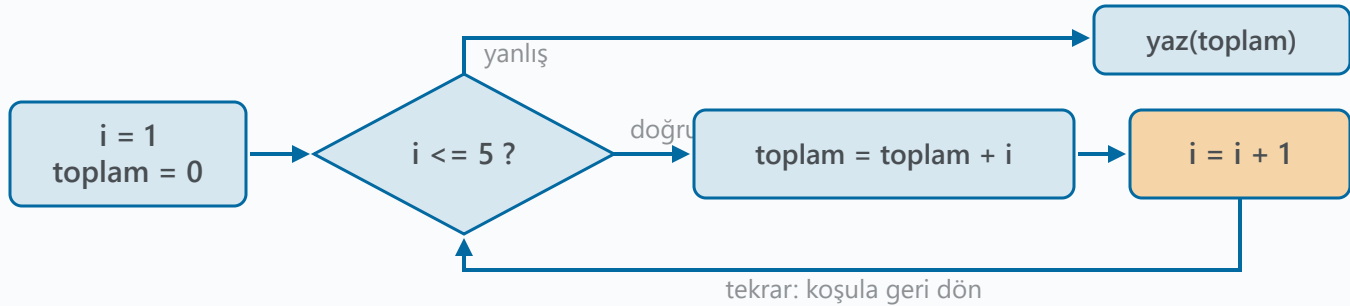
DEĞİŞKENLER

i = 2

toplam = 3

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



$i = 2 + 1 = 3$: sayaç ilerler.

Adım 8 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

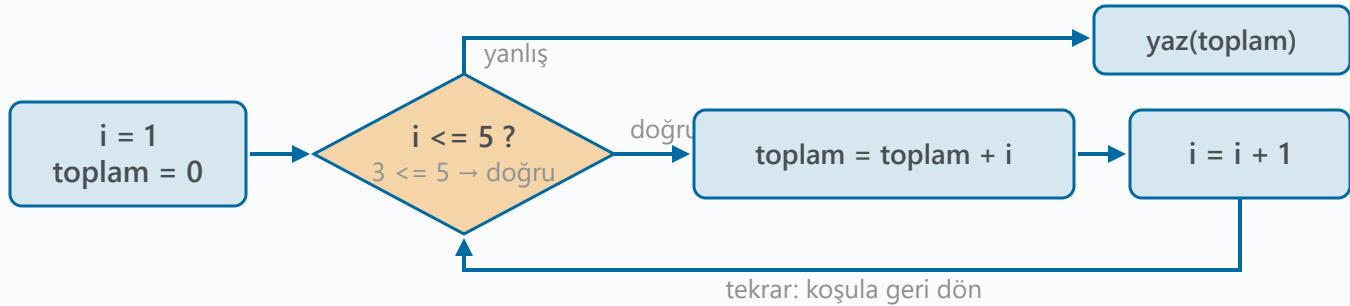
DEĞİŞKENLER

i = 3

toplam = 3

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: **3 <= 5** → **doğru**, 3. tur.

Adım 9 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

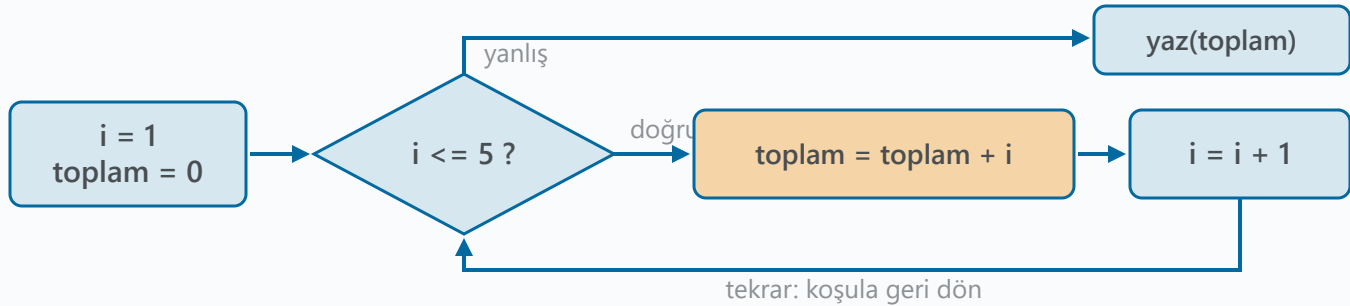
i = 3

toplam = 3

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



toplam = 3 + 3 = 6

Adım 10 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

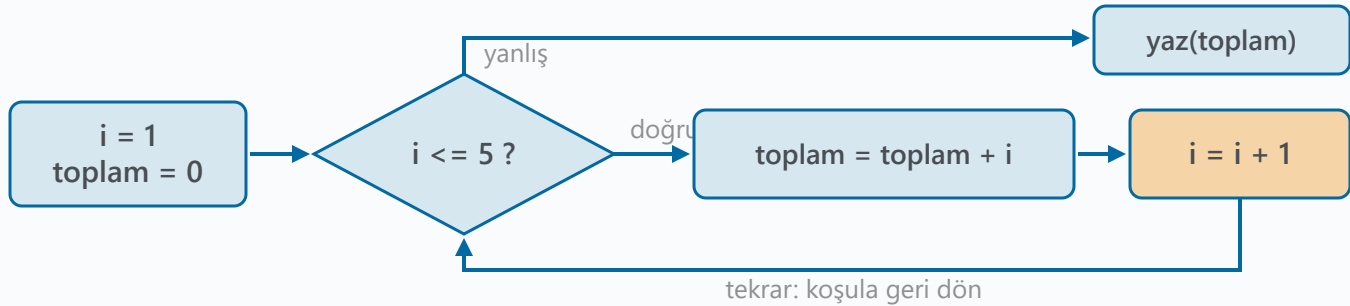
DEĞİŞKENLER

i = 3

toplam = 6

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



$i = 3 + 1 = 4$: sayaç ilerler.

Adım 11 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

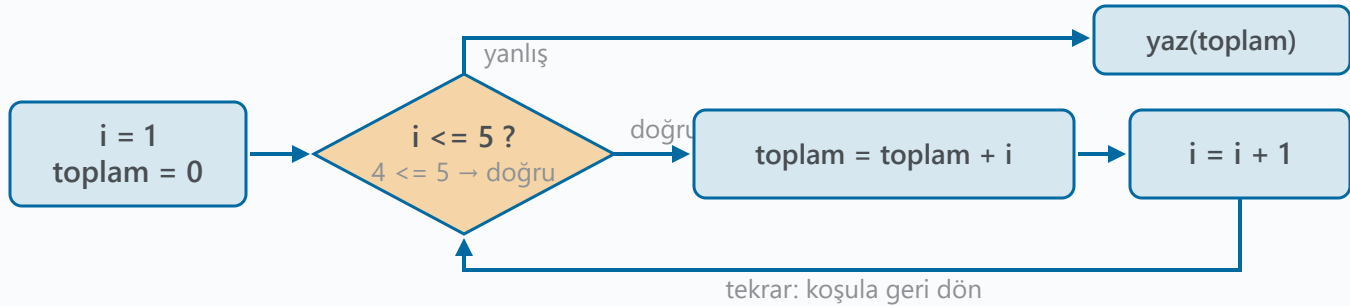
DEĞİŞKENLER

$i = 4$

$toplam = 6$

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: `4 <= 5` → **doğru**, 4. tur.

Adım 12 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

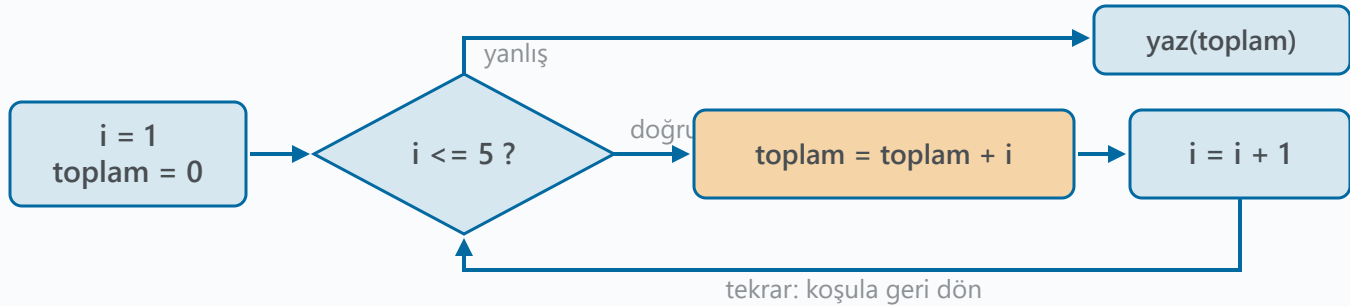
i = 4

toplam = 6

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



toplam = 6 + 4 = 10

Adım 13 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

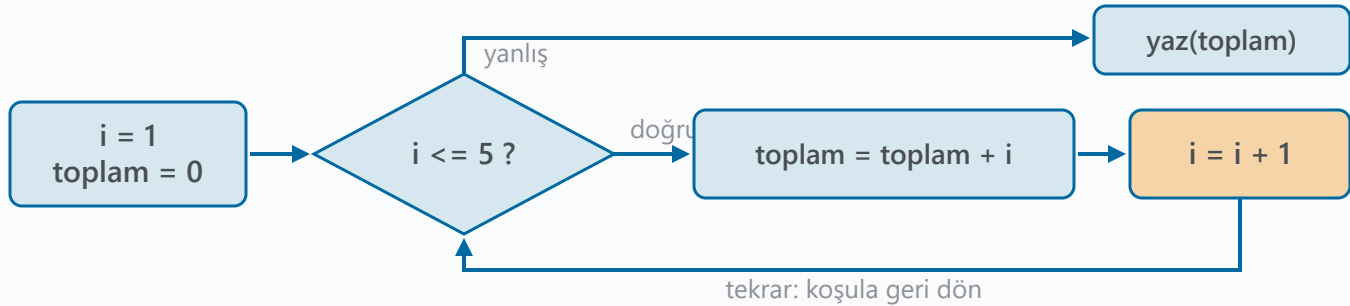
i = 4

toplam = 10

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



i = 4 + 1 = 5: sayaç ilerler.

Adım 14 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

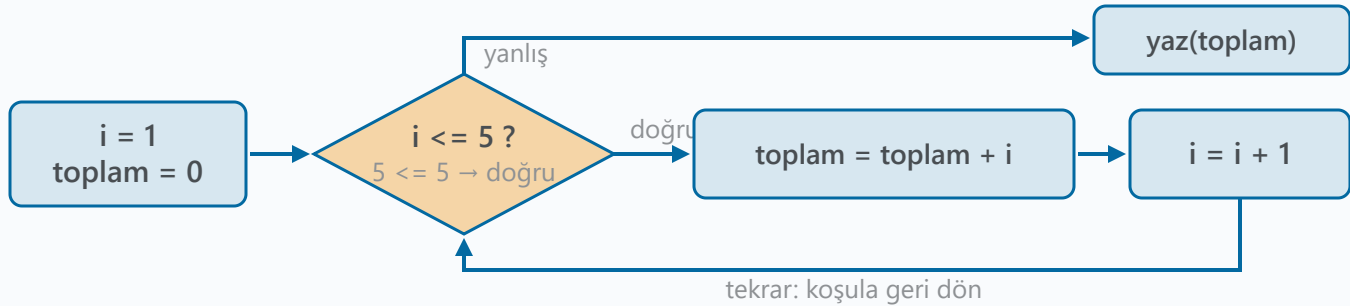
DEĞİŞKENLER

i = 5

toplam = 10

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: **5 <= 5** → **doğru**, 5. tur.

Adım 15 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

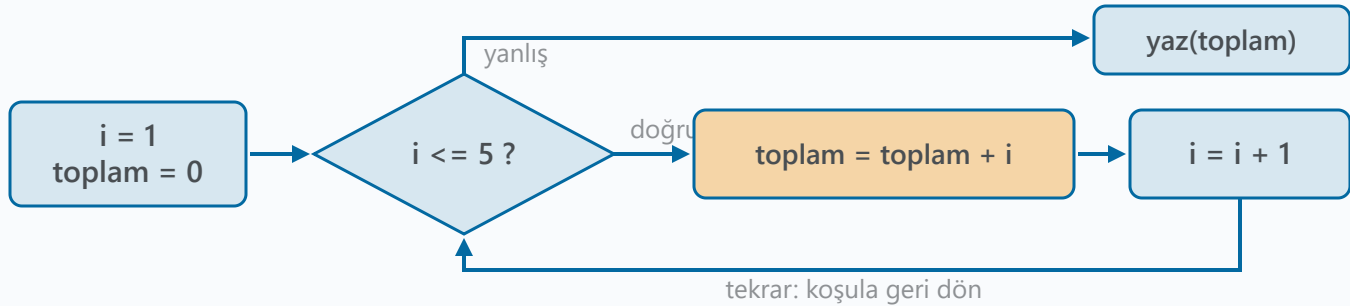
DEĞİŞKENLER

i = 5

toplam = 10

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



toplam = 10 + 5 = 15

Adım 16 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

DEĞİŞKENLER

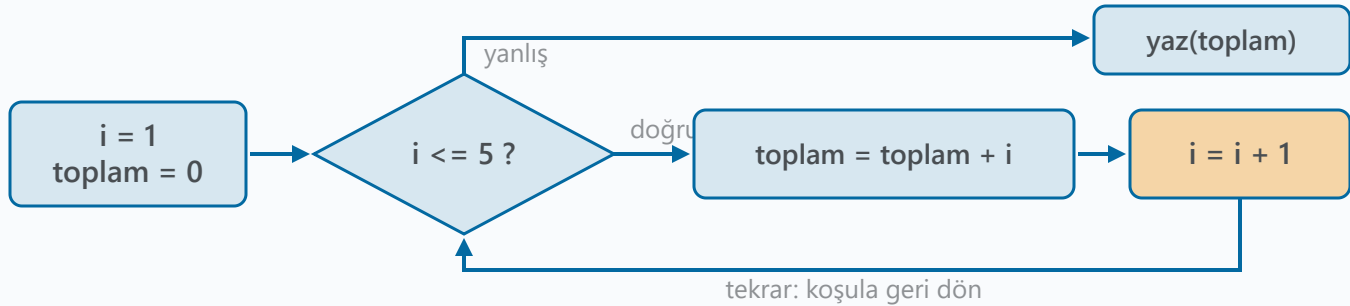
i = 5

toplam = 15

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



`i = 5 + 1 = 6`: sayaç ilerler.

Adım 17 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

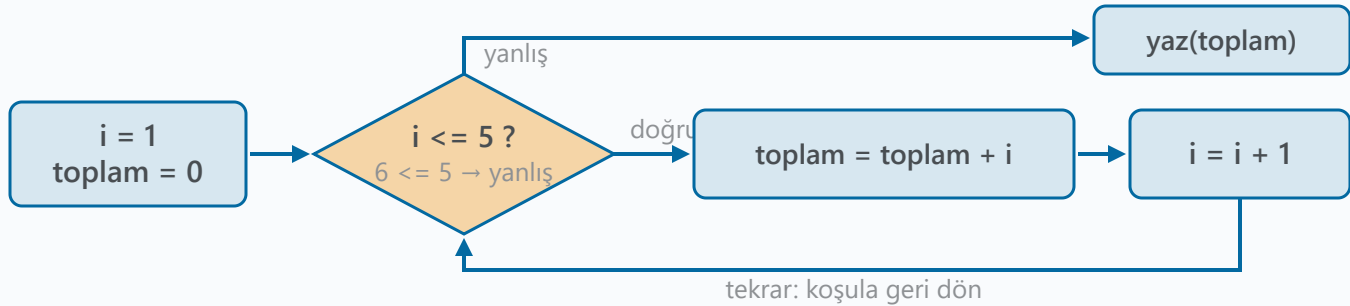
DEĞİŞKENLER

`i = 6`

`toplam = 15`

ÇIKTI

Döngü Adım Adım: 1'den 5'e Toplam



Koşul: **6 <= 5** → **yanlış**: döngüden çıkılır.

Adım 18 / 19

```
1 i = 1
2 toplam = 0
▶ while i <= 5
4     toplam = toplam + i
5     i = i + 1
6 yaz(toplam)
```

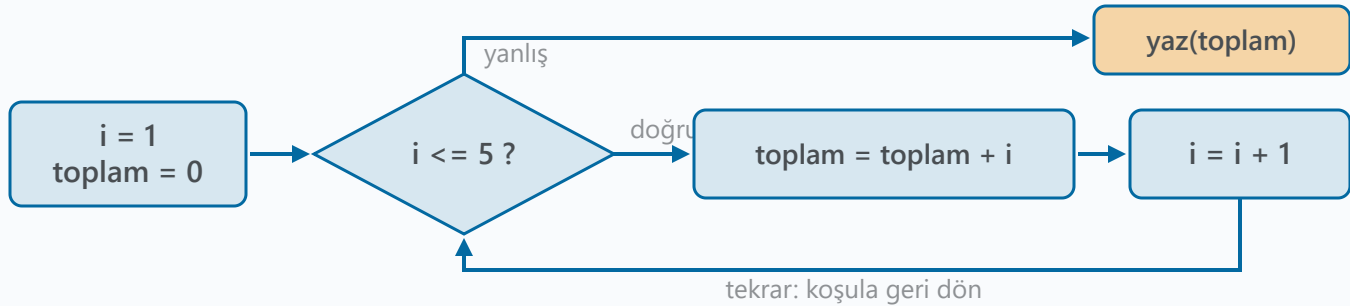
DEĞİŞKENLER

i = 6 toplam = 15

ÇIKTI



Döngü Adım Adım: 1'den 5'e Toplam



yaz(toplam) → 15 = 1+2+3+4+5. Gövde 5 kez, koşul 6 kez çalıştı.

Adım 19 / 19

```
1 i = 1
2 toplam = 0
3 while i <= 5
4     toplam = toplam + i
5     i = i + 1
▶ yaz(toplam)
```

DEĞİŞKENLER

i = 6

toplam = 15

ÇIKTI

15

Örnek: Dikdörtgen Alanı

BAŞLA

taban değerini oku

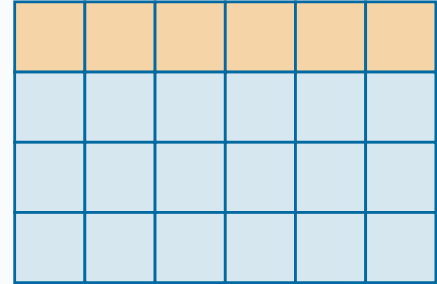
yukseklik değerini oku

$\text{alan} = \text{taban} * \text{yukseklik}$

alan değerini yaz

BİTİR

yukseklik = 4



taban = 6

$\text{alan} = 6 \times 4 = 24$ birim kare

(her satırda 6 kare, 4 satır)

Girdi: taban, yukseklik · İşlem: çarpma · Çıktı: alan

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

15

ikinci

?

Yeni girdi çifti: **ilk** = 15 okunur.

Adım 1 / 14

► ilk sayıyı oku

2 ikinci sayıyı oku

3 **eğer** ilk > ikinci **ise**

4 ilk sayıyı yaz

5 **aksi halde**

6 ikinci sayıyı yaz

DEĞİŞKENLER

ilk = 15

ÇIKTI

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

15

ikinci

4

ikinci = 4 okunur.

Adım 2 / 14

```
1 ilk sayıyı oku
2 ► ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

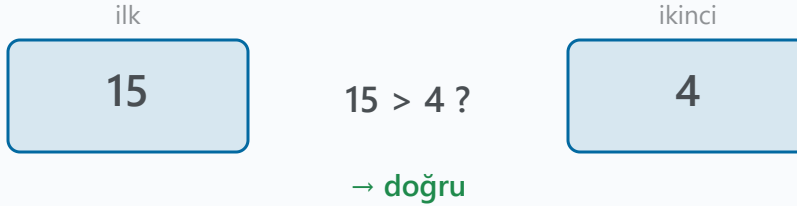
ilk = 15

ikinci = 4

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Koşul: **15 > 4** → **doğru**.

Adım 3 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
▶ eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 15

ikinci = 4

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Doğru dal: **15** yazılır, aksi halde dalı atlanır.

Adım 4 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 15

ikinci = 4

ÇIKTI

15

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

7

ikinci

?

Yeni girdi çifti: **ilk** = 7 okunur.

Adım 5 / 14

► ilk sayıyı oku

2 ikinci sayıyı oku

3 **eğer** ilk > ikinci **ise**

4 ilk sayıyı yaz

5 **aksi halde**

6 ikinci sayıyı yaz

DEĞİŞKENLER

ilk = 7

ÇIKTI

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

7

ikinci

12

ikinci = 12 okunur.

Adım 6 / 14

```
1 ilk sayıyı oku
2 ► ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 7

ikinci = 12

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Koşul: **7 > 12** → yanlış.

Adım 7 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
▶ eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 7

ikinci = 12

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Koşul yanlış → **aksi halde** dalına geçilir.

Adım 8 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 ▶ aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 7

ikinci = 12

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



12 yazılır.

Adım 9 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
   ► ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 7

ikinci = 12

ÇIKTI

12

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

9

ikinci

?

Yeni girdi çifti: **ilk** = 9 okunur.

Adım 10 / 14

► ilk sayıyı oku

2 ikinci sayıyı oku

3 **eğer** ilk > ikinci **ise**

4 ilk sayıyı yaz

5 **aksi halde**

6 ikinci sayıyı yaz

DEĞİŞKENLER

ilk = 9

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)

ilk

9

ikinci

9

ikinci = 9 okunur.

Adım 11 / 14

```
1 ilk sayıyı oku
2 ► ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 9

ikinci = 9

ÇIKTI

Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Koşul: **9 > 9** → **yanlış**. Sayılar **eşit**: ">" doğru olmaz.

Adım 12 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
▶ eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 9

ikinci = 9

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



Koşul yanlış → **aksi halde** dalına geçilir.

Adım 13 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 ▶ aksi halde
6     ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 9

ikinci = 9

ÇIKTI



Örnek: Büyük Sayıyı Bulma (15, 4 · 7, 12 · 9, 9)



9 yazılır: sonuç yine doğru ama eşitlik ayrıca belirtilmedi. "Eşit" iletisi isteniyorsa üçüncü durum eklenmelidir.

Adım 14 / 14

```
1 ilk sayıyı oku
2 ikinci sayıyı oku
3 eğer ilk > ikinci ise
4     ilk sayıyı yaz
5 aksi halde
   ► ikinci sayıyı yaz
```

DEĞİŞKENLER

ilk = 9

ikinci = 9

ÇIKTI

9

Örnek: Listedeki En Büyük Değer

```
enBuyuk = listenin ilk ögesi  
her öge için  
    eğer öge > enBuyuk ise  
        enBuyuk = öge  
enBuyuk değerini yaz
```

liste	-3	-7	-1	-5	satırlar: her öğeden sonra enBuyuk
ilk öge ile başla	-3	-3	-1	-1	→ sonuç -1 ✓ doğru
0 ile başla	0	0	0	0	→ sonuç 0 ✗ listede yok

- Başlangıç değeri 0 seçilirse yalnızca negatif sayılardan oluşan listede hata oluşabilir.
- İlk öğeyle başlamak bu varsayımı ortadan kaldırır.
- Algoritma her öğeyi bir kez inceler.

En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



enBuyuk listenin ilk ögesiyle başlar: **-3**. (0 ile başlasaydık tüm ögeler negatif olduğu için yanlış sonuç çıkardı.)

Adım 1 / 12

► **enBuyuk = listenin ilk ögesi**

2 **her** öge **için**

3 **eğer** öge > enBuyuk **ise**

4 enBuyuk = öge

5 enBuyuk değerini yaz

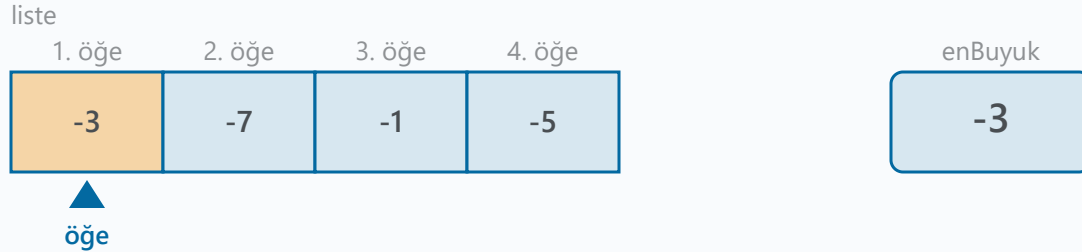
DEĞİŞKENLER

enBuyuk = -3

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Sıradaki öge: **-3** (1. öge).

Adım 2 / 12

```
1 enBuyuk = listenin ilk ögesi
```

```
► her öge için
```

```
3   eğer öge > enBuyuk ise
```

```
4       enBuyuk = öge
```

```
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

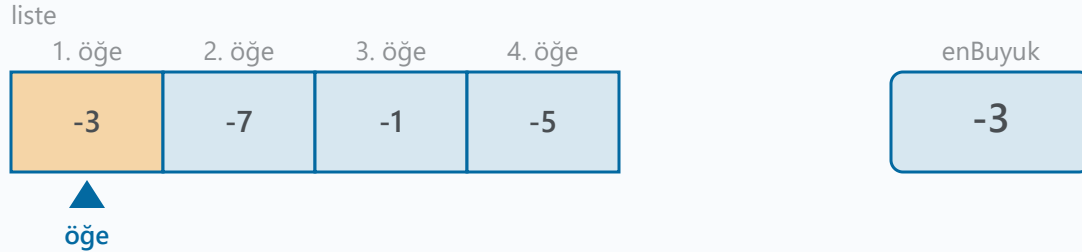
enBuyuk = **-3**

öge = **-3**

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Koşul: $-3 > -3 \rightarrow$ **yanlış**; enBuyuk değişmez.

Adım 3 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3   ► eğer öge > enBuyuk ise
4     enBuyuk = öge
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

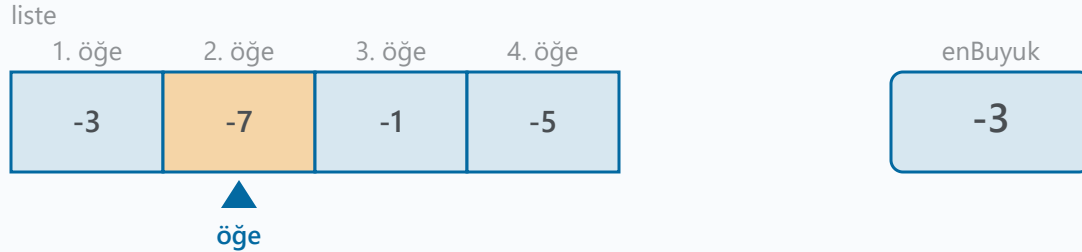
enBuyuk = -3

öge = -3

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Sıradaki öge: **-7** (2. öge).

Adım 4 / 12

```
1 enBuyuk = listenin ilk ögesi
```

```
► her öge için
```

```
3   eğer öge > enBuyuk ise
```

```
4       enBuyuk = öge
```

```
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

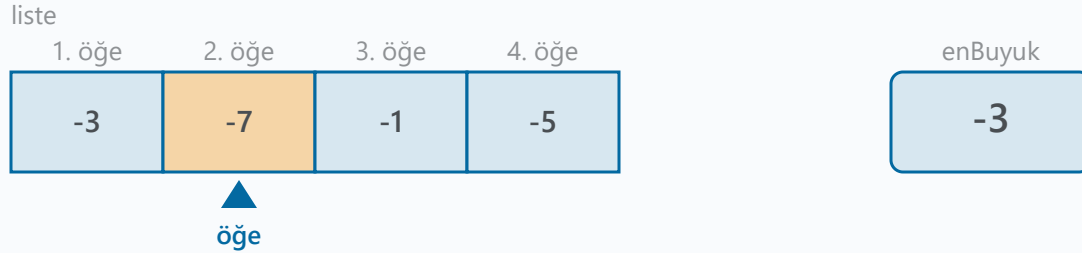
enBuyuk = -3

öge = -7

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Koşul: $-7 > -3 \rightarrow$ **yanlış**; enBuyuk değişmez.

Adım 5 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3     eğer öge > enBuyuk ise
4         enBuyuk = öge
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

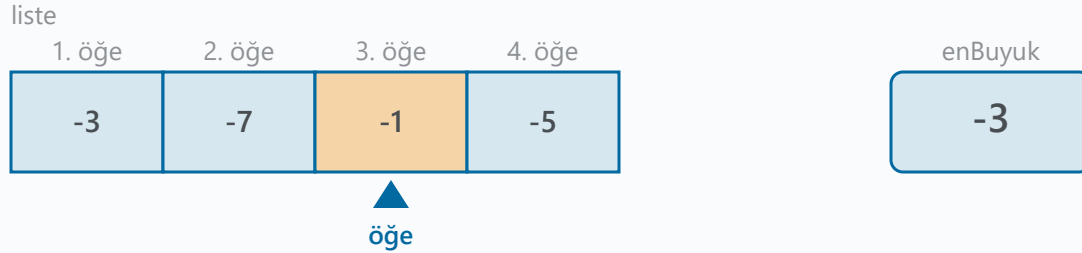
enBuyuk = -3

öge = -7

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Sıradaki öge: **-1** (3. öge).

Adım 6 / 12

```
1 enBuyuk = listenin ilk ögesi
```

```
► her öge için
```

```
3   eğer öge > enBuyuk ise
```

```
4       enBuyuk = öge
```

```
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

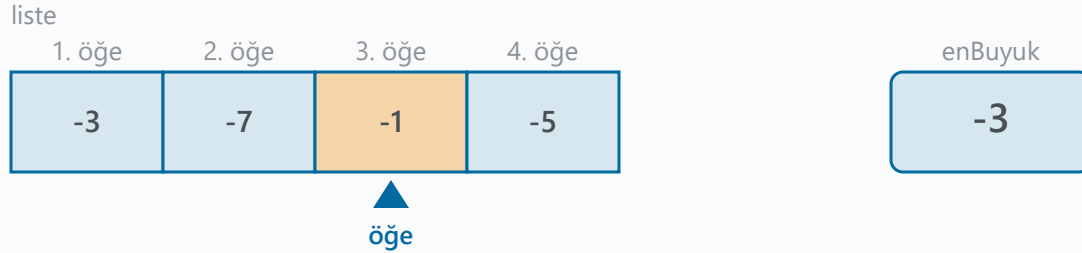
enBuyuk = -3

öge = -1

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Koşul: $-1 > -3 \rightarrow$ **doğru**.

Adım 7 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3   ► eğer öge > enBuyuk ise
4     enBuyuk = öge
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

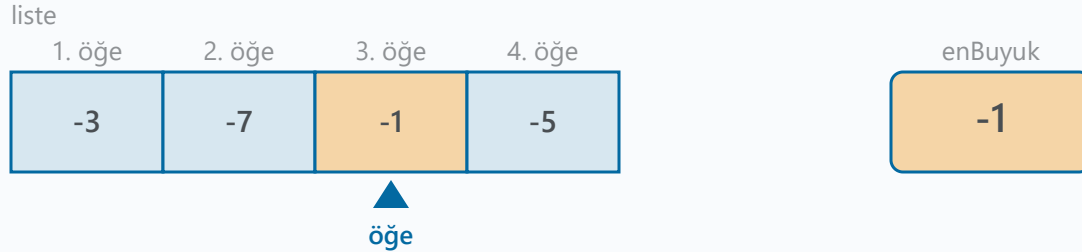
enBuyuk = -3

öge = -1

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Daha büyük bulundu: **enBuyuk** = -1.

Adım 8 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3   eğer öge > enBuyuk ise
4     ► enBuyuk = öge
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

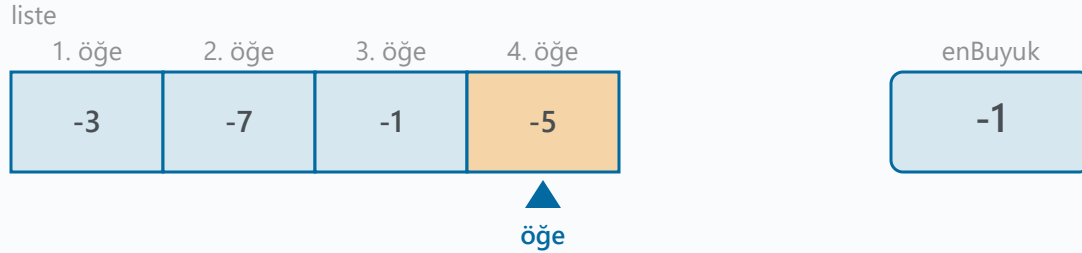
enBuyuk = -1

öge = -1

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Sıradaki öge: **-5** (4. öge).

Adım 9 / 12

```
1 enBuyuk = listenin ilk ögesi
```

```
► her öge için
```

```
3   eğer öge > enBuyuk ise
```

```
4       enBuyuk = öge
```

```
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

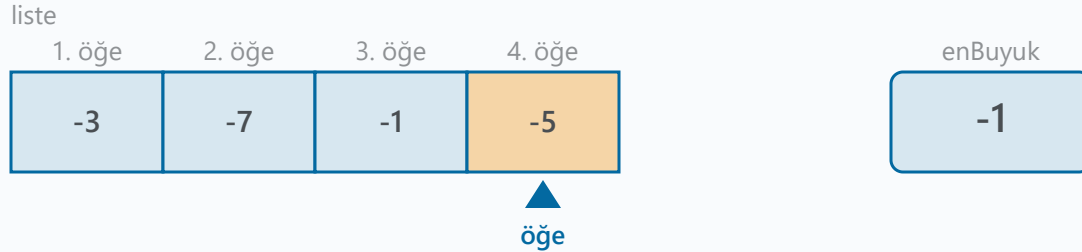
enBuyuk = -1

öge = -5

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Koşul: $-5 > -1 \rightarrow$ **yanlış**; enBuyuk değişmez.

Adım 10 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3     ► eğer öge > enBuyuk ise
4         enBuyuk = öge
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

enBuyuk = -1

öge = -5

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$

liste

1. öge

2. öge

3. öge

4. öge

-3	-7	-1	-5
----	----	----	----

enBuyuk

-1

Listede öge kalmadı; döngü biter.

Adım 11 / 12

```
1 enBuyuk = listenin ilk ögesi
```

```
► her öge için
```

```
3   eğer öge > enBuyuk ise
```

```
4       enBuyuk = öge
```

```
5 enBuyuk değerini yaz
```

DEĞİŞKENLER

```
enBuyuk = -1
```

ÇIKTI



En Büyük Değer Adım Adım: $[-3, -7, -1, -5]$



Sonuç **-1**. Her öge **bir kez** incelendi: 4 öge → 4 karşılaştırma.

Adım 12 / 12

```
1 enBuyuk = listenin ilk ögesi
2 her öge için
3     eğer öge > enBuyuk ise
4         enBuyuk = öge
▶ enBuyuk değerini yaz
```

DEĞİŞKENLER

enBuyuk = **-1**

ÇIKTI

-1

Aynı Problem, İki Algoritma: $1 + 2 + \dots + n$

```
toplam = 0
i = 1
while i <= n
    toplam = toplam + i
    i = i + 1
```

A: döngü, n tur

```
toplam = n * (n + 1) / 2
```

B: Gauss formülü, tek adım

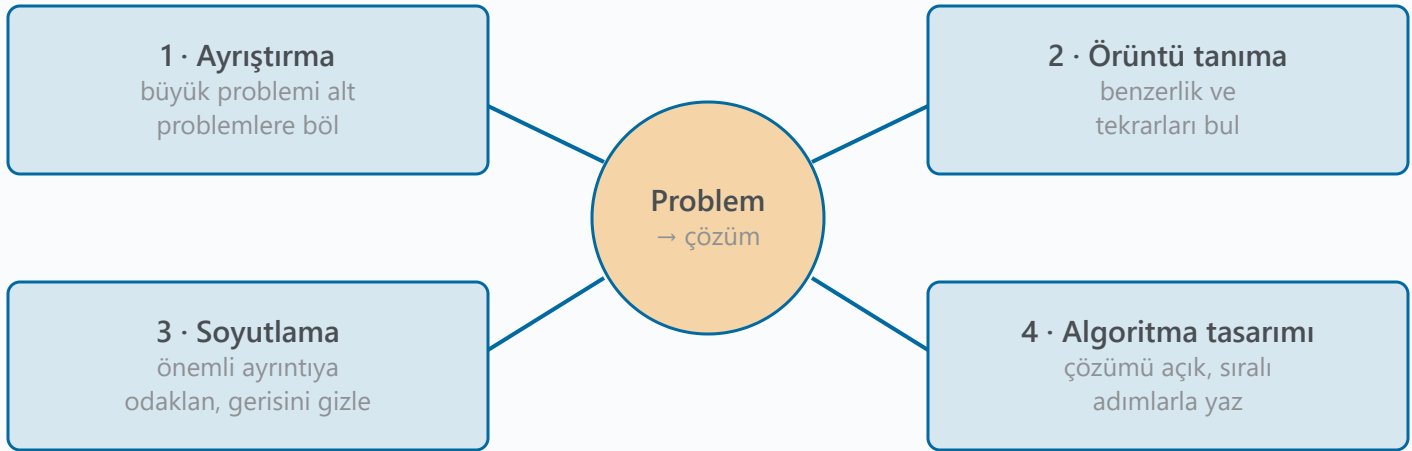
Toplama işlemi sayısı

n = 10	döngü: 10 toplama
	formül: 1 çarpma + 1 bölme · sonuç 55
n = 100	döngü: 100 toplama
	formül: 1 çarpma + 1 bölme · sonuç 5050
n = 1000	döngü: 1000 toplama
	formül: 1 çarpma + 1 bölme · sonuç 500500

İkisi de **doğru**; ama n büyüdükçe A'nın işi artar, B'ninki sabit kalır: doğruluk kadar **verimlilik** de önemlidir.

Hesaplamalı Düşünme

Hesaplamalı düşünme, problemleri bilgisayarın veya insanın sistemli biçimde çözebileceği yapılara dönüştürme yaklaşımıdır.



Ayrıştırma

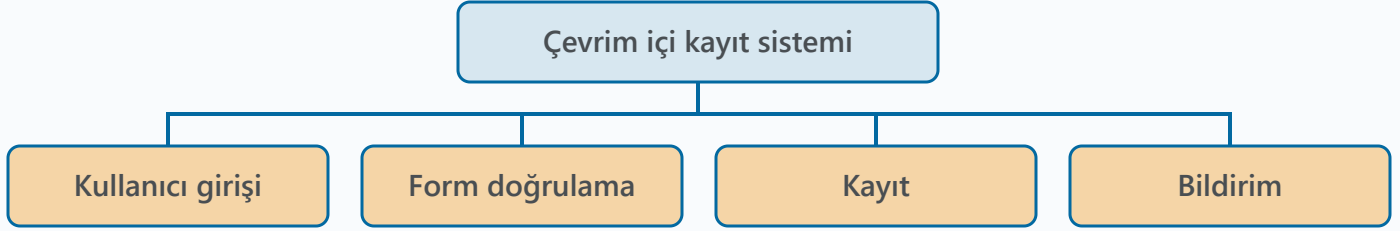
Çevrim içi kayıt sistemi

Büyük problem: bir **çevrim içi kayıt sistemi**. Tek parça olarak ele almak zor.

Adım 1 / 3

- Her alt problem ayrı geliştirilebilir ve test edilebilir.
- Alt problemlerin girdi ve çıktıları açıkça tanımlanır.
- Bağımlılıklar erken fark edilir.

Ayrıştırma



Ana alt görevlere bölünür; her birinin girdisi ve çıktısı ayrı tanımlanır.

Adım 2 / 3

- Her alt problem ayrı geliştirilebilir ve test edilebilir.
- Alt problemlerin girdi ve çıktıları açıkça tanımlanır.
- Bağımlılıklar erken fark edilir.

Ayrıştırma



Gerekirse daha da bölünür. Her yaprak küçük, tek başına yazılıp test edilebilir bir iştir.

Adım 3 / 3

- Her alt problem ayrı geliştirilebilir ve test edilebilir.
- Alt problemlerin girdi ve çıktıları açıkça tanımlanır.
- Bağımlılıklar erken fark edilir.

Örüntü ve Soyutlama

Öğrenci
ad
kimlik no
e-posta
öğrenci no

Personel
ad
kimlik no
e-posta
birim

Ziyaretçi
ad
kimlik no
e-posta
ziyaret tarihi

Üç farklı kayıt türü: öğrenci, personel ve ziyaretçi.

Adım 1 / 3

İyi soyutlama, önemli farkları kaybetmeden tekrar eden yapıyı ortaklaştırır.

Örüntü ve Soyutlama

Öğrenci
ad
kimlik no
e-posta
öğrenci no

Personel
ad
kimlik no
e-posta
birim

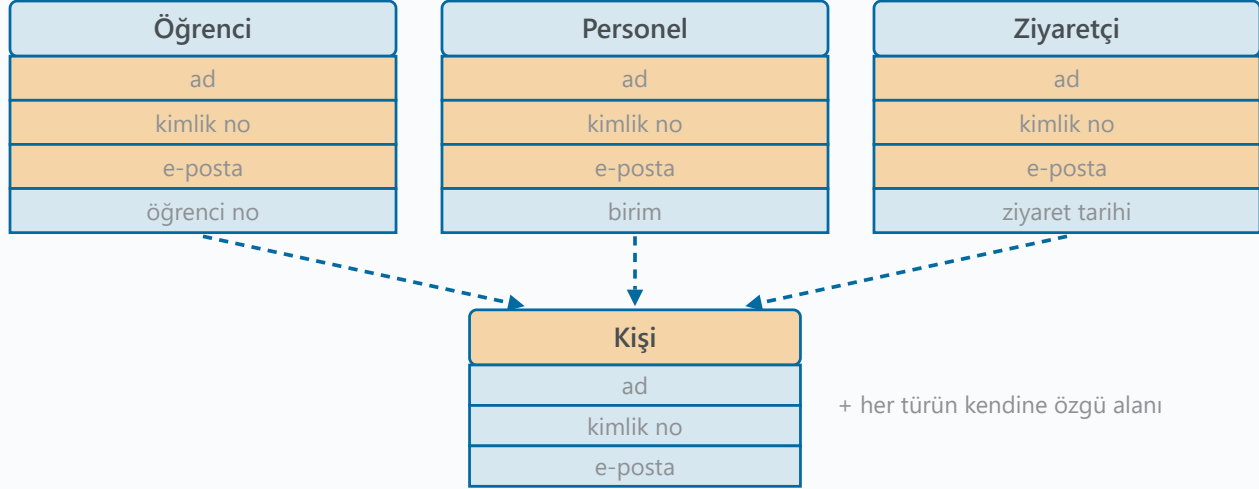
Ziyaretçi
ad
kimlik no
e-posta
ziyaret tarihi

Örüntü tanıma: ad, kimlik no ve e-posta alanları üçünde de tekrar ediyor.

Adım 2 / 3

İyi soyutlama, önemli farkları kaybetmeden tekrar eden yapıyı ortaklaştırır.

Örüntü ve Soyutlama

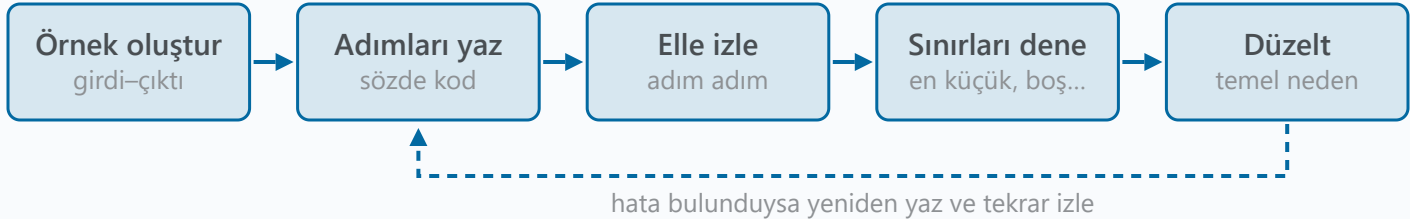


Soyutlama: ortak alanlar tek bir **Kişi** modelinde toplanır; farklı ayrıntılar yalnızca gereken türde kalır.

Adım 3 / 3

İyi soyutlama, önemli farkları kaybetmeden tekrar eden yapıyı ortaklaştırır.

Algoritma Tasarımı ve Test



- Normal, en küçük, en büyük ve geçersiz girdiler düşünülür.
- Her adımın beklenen durumu ürettiği izlenir.
- Hata bulunduğunda yalnızca belirti değil, temel neden düzeltilir.

Bölüm Özeti

- Programlama, algoritmaları bilgisayarın uygulayabileceği komutlara dönüştürür.
- Algoritma açık, sonlu, doğru ve uygulanabilir adımlardan oluşur.
- Problem tanımı; amaç, girdi, çıktı ve kısıtları içerir.
- Sıralı, koşullu ve döngülü yapılar program akışını belirler.
- Hesaplamalı düşünme; ayrıştırma, örüntü tanıma, soyutlama ve algoritma tasarımını birleştirir.
- Zaman, bellek, enerji ve hesaplanabilirlik bilgisayarların sınırlarını oluşturur.

Kontrol Soruları

1. Hesaplama neden yalnızca aritmetik değildir?
2. Bildirimsel ve buyurgan bilgiye birer örnek verin.
3. İyi bir algoritmanın temel özellikleri nelerdir?
4. Sözde kod neden programlama dilinden önce kullanılabilir?
5. Koşul ile döngü arasındaki temel fark nedir?
6. Bir problemi hesaplamalı düşünmenin dört bileşeniyle nasıl ele alırsınız?

Alıştırmalar

1. **GCD(84, 36)**'yı Öklid algoritmasıyla izleyin; her turda a , b , r değerlerini tabloya yazın.
2. Üç sayının en büyüğünü bulan sözde kodu yazın (`a` , `b` , `c` , `enBuyuk`); (4, 9, 9) girdisiyle adım adım izleyin.
3. 1'den n 'e kadar **çift** sayıların toplamını bulan döngülü sözde kod yazın; $n = 6$ için her turda `i` ve `toplam` değerlerini gösterin.
4. Sayıya 1 ekleyen Turing makinesini bantta **1011** ile (başlık en sağda) çalıştırın; bant hangi değerle durur?
5. Karekök yönteminde $x = 10$ ve tahmin = 1 ile üç tur yapın; tahmin hangi değere yaklaşıyor?
6. Bir **kütüphane ödünç alma sistemini** ayrıştırın: en az dört alt görev ve her birinin girdi/çıktısı; hangi alanlar üye ve personel kayıtlarında ortaktır?

Kaynaklar

- Wing, Computational Thinking, Communications of the ACM, 2006.
- Cormen ve diğerleri, Introduction to Algorithms.
- Downey, Think Java.
- Sipser, Introduction to the Theory of Computation.

Son

Hesaplama ve Hesaplamalı Düşünme

Sonraki sunum: [Yazılım ve İşletim Sistemleri](#) · [Çalışma soruları](#) · Simülatör: [Turing Makinesi](#) · [Terim sözlüğü](#)